Agentic Software Issue Resolution with Large Language Models: A Survey

ZHONGHAO JIANG, College of Computer Science and Technology and the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China
DAVID LO, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore
ZHONGXIN LIU*, College of Computer Science and Technology and the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China

Software issue resolution task aims to address real-world issues in software repositories based on natural language descriptions provided by users, representing a key aspect of software maintenance. With the rapid development of large language models (LLMs) in reasoning and generative capabilities, LLM-based approaches have made significant progress in automated software issue resolution. However, real-world software issue resolution is inherently complex and requires long-horizon reasoning, iterative exploration, and feedback-driven decision-making that demand agentic capabilities beyond conventional single-step approaches. Recently, LLM-based agentic systems have become a promising research direction for software issue resolution, since the related literature has experienced explosive growth. Advancements in agentic software issue resolution can not only greatly enhance software maintenance efficiency and quality but also provide a realistic environment for validating agentic systems' reasoning, planning, and execution capabilities, bridging AI and software engineering. This work presents a systematic survey of 242 recent studies at the forefront of LLM-based agentic software issue resolution research. It outlines the general workflow of the task and establishes a taxonomy across three dimensions: benchmarks, techniques, and empirical studies. Furthermore, it highlights how reinforcement learning has become an increasingly important training paradigm for agentic systems in software engineering. Finally, it summarizes key challenges and outlines promising directions for future research. The artifacts' page accompanying this survey is at https://github.com/ZhonghaoJiang/Awesome-Issue-Solving.

CCS Concepts: • **General and reference → Surveys and overviews**; • **Software and its engineering → Maintaining software**; *Automatic programming*; • **Computing methodologies → Intelligent agents**; *Natural language processing*.

Additional Key Words and Phrases: Software issue resolution, large language models, LLM-based agents

1 Introduction

In software engineering, software maintenance often accounts for about two-thirds of software lifecycle costs [121]. Software issue resolution occupies a crucial position in software maintenance [104]. This task aims to understand, locate, and resolve issues in real-world code repositories based on developers' natural language descriptions of issues, covering diverse maintenance activities such as bug fixes, feature additions, and efficiency optimizations. Hereafter, we refer to this task as issue resolution for brevity. Traditional issue resolution methods heavily rely

*Corresponding author.

Authors' Contact Information: Zhonghao Jiang, College of Computer Science and Technology and the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: zhonghao.j@zju.edu.cn; David Lo, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: davidlo@smu.edu.sg; Zhongxin Liu, College of Computer Science and Technology and the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, Zhejiang, China; e-mail: liu_zx@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 1557-7341/2026/9-ART
https://doi.org/10.1145/3849702

Just Accepted

on human expertise, making the process time-consuming, error-prone, and difficult to scale [168]. Consequently, issue resolution has long been regarded as a critical bottleneck to efficient software evolution [16].

In recent years, LLMs have achieved significant success in multiple areas of software engineering [17] due to their rapid advancements in code understanding [186], reasoning [63], and generation [86], paving the way for automated issue resolution. However, issue resolution in real-world software systems is inherently complex, typically involving long-horizon reasoning, iterative exploration, interaction with evolving codebases, and feedback-driven decision-making, which goes beyond the capabilities of conventional single-step automated approaches. LLM-based agentic systems are autonomous, goal-driven AI architectures that can interpret objectives, plan multi-step tasks, and adapt their behavior based on environmental feedback [178]. They have become increasingly popular in recent issue resolution research and report the strongest results on public benchmarks [227, 236, 252]. For example, related papers on agentic issue resolution systems rose from 35 in 2024 to 122 in 2025, achieving a 248.6% increase. Moreover, recent agentic systems built on the latest frontier model Claude Fable 5 [10] can already resolve 80.3% of issues on the challenging SWE-Bench Pro benchmark [41]. On the other hand, because of the complexity and broad applicability, issue resolution has become a key task for evaluating LLMs and software engineering (SE) agentic systems [10]. Advancing LLM-based agentic issue resolution not only holds the potential to significantly improve the efficiency and quality of software maintenance but also provides LLM-based agentic systems with a realistic environment to validate complex reasoning, planning, and execution capabilities, which serves as a crucial bridge for the bidirectional integration of AI and software engineering. Given its significance and impact, a systematic survey of LLM-based agentic issue resolution is essential to provide a comprehensive overview of current research and to highlight future directions.

Although existing surveys [64, 123, 206, 250] have organized a part of agentic issue resolution methods based on the taxonomy of automated program repair (APR) or code generation, they have not comprehensively covered the literature in the field of issue resolution for the following reasons. ❶ Lack of taxonomies specifically tailored to issue resolution and workflow modeling. Existing surveys [64, 206, 250] do not regard issue resolution as a holistic task but divide issue resolution-related research into existing areas, such as APR. However, unlike APR, issue resolution encompasses more diverse software maintenance activities, such as efficiency optimization [75] and feature addition [40, 88, 116]. Even for bug-fixing issues, issue resolution does not assume the existence of tests that can trigger the bug. These differences render existing taxonomies insufficient to systematically organize all research related to issue resolution. Additionally, these surveys also ignore the examination of numerous issue resolution benchmarks and empirical studies. ❷ Overlook the recent paradigm shift in issue resolution techniques. Recently, LLM-based issue resolution has gradually shifted from prompt-engineering-based scaffold design toward training dedicated models. For instance, since February 2025, the number of training studies has grown 11.4-fold, substantially outpacing the 3.1-fold growth of scaffold-design studies over the same period. Within this training-driven paradigm, reinforcement learning (RL) has emerged as the main approach, propelled by its remarkable success in LLMs [63, 269]. 24 of the 39 (61.5%) representative models released from February 2025 to May 2026 use reinforcement learning. Existing surveys [123, 250] did not capture this paradigm shift, overlooking the studies on model training strategies for issue resolution. In summary, there is still no comprehensive survey on LLM-based agentic issue resolution.

To fill this gap, we present a comprehensive survey of LLM-based agentic issue resolution, aiming to provide researchers with a foundational reference for quickly understanding this emerging area. Specifically, we mainly focus on software issue resolution, treating the end-to-end methods as the core task, the subtask-level methods (e.g., issue localization), the learning strategies (e.g., RL training) as part of this problem space, the benchmark construction as evaluation infrastructure, and the empirical studies as the supporting findings. Based on this scope, we propose a taxonomy that spans three dimensions: benchmarks, techniques, and empirical studies, offering an analysis from task definition of issue resolution to solutions and reflections. In particular, under the technical dimension, we examine each phase of the issue resolution process, including localization, repair, and

Just Accepted

validation from two perspectives: solution design paradigms and learning strategies for domain-specific models. This highlights how the rise of reinforcement learning has brought a significant transformation to the field. Furthermore, we identify key challenges and future opportunities in the current landscape, making this survey a valuable resource for researchers in both the software engineering and the natural language processing (NLP) communities to grasp the research trajectory, frontiers, limitations, and prospects of the issue resolution task. *To the best of our knowledge, this is the first survey that specifically focuses on LLM-based agentic issue resolution tasks.*

The remainder of this survey is organized as follows. Section 2 introduces the background of automatic issue resolution, LLM-based agentic systems, and related surveys. Section 3 defines research questions to guide the analysis and outlines the methodology for systematically conducting this survey. Section 4 presents the construction and evolution of current issue resolution benchmarks. Section 5 provides a detailed taxonomy of existing techniques from the perspectives of method design and learning strategies. Section 6 categorizes and analyzes empirical studies related to the issue resolution task. Section 7 discusses potential research opportunities and future directions. Section 8 analyzes the threats to the validity of this survey. Finally, Section 9 concludes this survey.

2 Background

2.1 LLM-based Agentic Systems for Issue Resolution

The automated, *end-to-end* issue resolution task takes a natural-language issue description and a code repository as input and produces a repository-level patch resolving the issue. From the functional components of the surveyed systems, we abstract this task into five phases, namely repo preprocessing, localization, repair, patch validation, and patch selection, whose coverage is validated by the open coding procedure described in Section 3.3. These phases denote logical functions rather than a fixed execution order, so a system may execute them sequentially or interleave them dynamically at inference time. Appendix A illustrates the resulting framework and describes each phase in detail.

LLM-based agentic systems for issue resolution are composed of LLMs and scaffolds [232]. The LLM serves as the core reasoning and generation engine of the system, responsible for understanding natural-language issue descriptions, analyzing code semantics, and generating patches or intermediate reasoning steps. In contrast, scaffolds provide external structured control and an operational framework by orchestrating task workflows, coordinating multi-step reasoning, and invoking tools for code retrieval and execution, thereby guiding the LLM through iterative refinement and enabling the system to resolve the issue in real-world software repositories.

Existing agentic systems can be divided into agentic pipelines and agents based on how their control flow is designed and executed [178]. In pipelines, human designers predefine the control flow as staged steps or state-machine-like workflows, while the LLM performs the assigned subtask at each stage. In agents, the LLM determines the next action during execution based on environmental feedback, so the control flow emerges dynamically during execution. In summary, pipelines emphasize determinism and controllability, whereas agents offer greater autonomy and flexibility.

2.2 Related Surveys

We mainly compare surveys [64, 123, 206, 250] conducted since the introduction of automated issue resolution [90] from several high-level issue-resolution-related dimensions, as shown in Table 1. Overall, existing surveys do not examine issue resolution as a distinct area. Instead, they discuss it primarily in the context of related tasks, such as automated program repair (APR) or repository-level code generation, covering only the aspects that fall within their respective scopes. For example, Liu et al. [123] focus on the design of agent modules for general software engineering tasks and describe issue resolution only as a coarse maintenance process, while overlooking the training strategies and issue-resolution-specific empirical findings. Yang et al. [250] organize issue resolution

Just Accepted

Table 1. Comparison of related surveys. ✓: systematic coverage; ❍: partial coverage; ✗: not covered.

Dimension	Liu et al. [123]	Yang et al. [250]	Tao et al. [206]	Guo et al. [64]	Ours
Primary Scope	**SE agents**	**LLM-based APR**	**Repository-level RACG**	**Agentic SE**	**Issue resolution**
Workflow Modeling	❍ Maintenance Process	✗	✗	❍ Agent Workflow	✓ Five Logical Task Phases
Scaffold Design	✓ Agent Modules	✓ APR Design Paradigms	✓ Agent Autonomy Levels	✓ Solution Paradigms	✓ Pipeline and Agent Scaffolds
Training Strategies	✗	❍ SFT and RL	❍ Retriever and Generator Training	❍ SFT and RL	✓ Data Preparation, SFT, RL
Benchmark Analysis	❍ Benchmark Listing	❍ Scope and Evaluation Settings	❍ Benchmark Listing	❍ Task-based Categories	✓ Construction and Evolution
Empirical Findings	✗	❍ Success-rate Trends	❍ RAG Effectiveness	✗	✓ Findings across All Phases
Collection	124 papers, until Sept. 2024	63 papers, Jan. 2022–Oct. 2025	110 papers, Jan. 2023–Aug. 2025	150+ papers, 2023–2025	242 papers, Oct. 2023–May 2026

methods under APR design paradigms and discuss supervised fine-tuning (SFT) and reinforcement learning (RL), benchmark settings, and success-rate trends only for APR, while overlooking non-bug-fixing issues and the fact that issue resolution assumes no trigger tests and thus follows a different workflow. Guo et al. [64] catalogue solutions, SFT and RL techniques, and task-based benchmarks for agentic software engineering, while modeling only a general agent workflow and overlooking benchmark construction and empirical findings. Tao et al. [206] concentrate on retrieval strategies, retriever and generator training, and the effectiveness of retrieval-augmented generation (RAG) for repository-level code generation, while overlooking the issue resolution workflow and the construction and evolution of its benchmarks.

Different from these surveys, our work treats issue resolution as the central research topic. Based on 242 collected papers, it models the five logical phases of the task, organizes techniques along both scaffold design and training strategies that cover data preparation, SFT, and RL, analyzes the construction and evolution of issue resolution benchmarks, and synthesizes empirical findings across all of these phases. Furthermore, it systematically examines how the task has evolved from its proposal to the latest developments, revealing the paradigm-level shift in the field and elucidating the technical landscape both before and after this transition.

3 Survey Methodology

This section details the systematic methodology employed for conducting the literature review. Following prior surveys [77, 86, 123], we adopt the systematic approach proposed by Kitchenham et al. [100] and Petersen et al. [163].

3.1 Research Questions

To guide our systematic literature review and characterize this field, we formulate four research questions (RQs), as follows:

- ***RQ1:*** *How is the issue resolution task evaluated?* This RQ investigates how the existing evaluation datasets are constructed and have evolved, as well as which metrics are used for evaluation.
- ***RQ2:*** *How are the key components in existing agentic issue resolution systems developed?* This RQ examines existing scaffolds, focusing on the design of their core components and the training of the LLMs that power them.
- ***RQ3:*** *What do empirical studies reveal about the trustworthiness of reported issue-resolution performance and the factors and processes underlying agent performance variation?* This RQ synthesizes empirical findings on the validity of existing evaluation results and the factors and mechanisms that cause agent performance to vary.
- ***RQ4:*** *What key challenges remain in LLM-based agentic issue resolution, and what research opportunities do they suggest?* This RQ identifies unresolved challenges in agentic software issue resolution, and outlines promising directions for future research.

Just Accepted

3.2 Literature Search and Selection

To answer the RQs presented above and perform a survey, we need to retain relevant studies published over a wide time range and conduct a comprehensive analysis on those relevant studies. To achieve this, we follow prior surveys and use a four-stage process comprising automated search, inclusion/exclusion screening, quality assessment, and backward/forward snowballing. Due to space limitations, we include a detailed description of each step in Appendix B.

3.3 Taxonomy Construction

To provide systematic answers to RQ1–RQ3, we aim to construct taxonomies for benchmarks, techniques, and empirical studies. These categories are derived from the primary research objectives of the included papers during the inclusion and exclusion stage. Following prior work [180], we construct all three taxonomies through the standard open coding procedure. Specifically, we randomly sample 70% of the papers of each study type to construct the initial taxonomies. Two authors jointly read these papers multiple times, inspect their abstracts, introductions, related work, and conclusions to understand their research goals, assign short phrases summarizing their contributions, group similar phrases into categories in a bottom-up manner, and iteratively refine the categories while revisiting the papers. If a paper is associated with more than one category, it is assigned to all relevant categories. The remaining 30% of the papers are then independently labeled by the two authors based on the initial taxonomies, and each paper is marked with the leaf categories of the corresponding taxonomy. Papers that cannot be classified into the current taxonomy are temporarily placed in a pending category and later discussed. We use Cohen's κ to evaluate inter-rater agreement during independent labeling, reaching values of 0.8682, 0.8985, and 0.9107 for the benchmark, technique, and empirical-study taxonomies, respectively. Disagreements in both phases, including pending cases, are resolved through discussion with a third author. As a result, every paper maps to at least one leaf category of the corresponding taxonomy. In addition, this procedure validates the coverage of the five-phase workflow model in Section 2.1. Functional components that directly support issue resolution are mapped to the corresponding phases of the resolution workflow. Cross-cutting mechanisms, such as memory, context management, and self-improvement, are treated separately because they operate across multiple phases.

4 Benchmarks

To answer **RQ1**, we systematically introduce the evaluation for software engineering agentic systems on the issue resolution task and its subtasks, focusing on three dimensions, including benchmark construction, benchmark evolution, and evaluation metrics, as shown in Figure 1. These three dimensions reflect the life cycle of a benchmark, covering how it is constructed, how it evolves after release, and how it measures performance. In addition, Section 4.4 examines benchmark validity and comparability to clarify what the reported evaluation scores represent.

4.1 Benchmark Construction

As shown in Table 4 (see Appendix C.1), based on the curation process of benchmark construction, we categorize benchmarks into *manual construction* and *automatic construction*. We also provide statistics on the number of repositories, number of instances, programming languages, system kind, domain, issue type, release time, and data sources involved in all benchmarks. This reveals clear cross-benchmark patterns. Early benchmarks concentrate on bug fixing over Python libraries, frameworks, and developer tools from the data science, machine learning, and web development domains, reflecting their shared SWE-bench lineage, while recent benchmarks diversify toward more programming languages, more system kinds such as end-user applications and heterogeneous open-source projects, broader domains such as mobile and enterprise software, and richer issue types including feature

Just Accepted

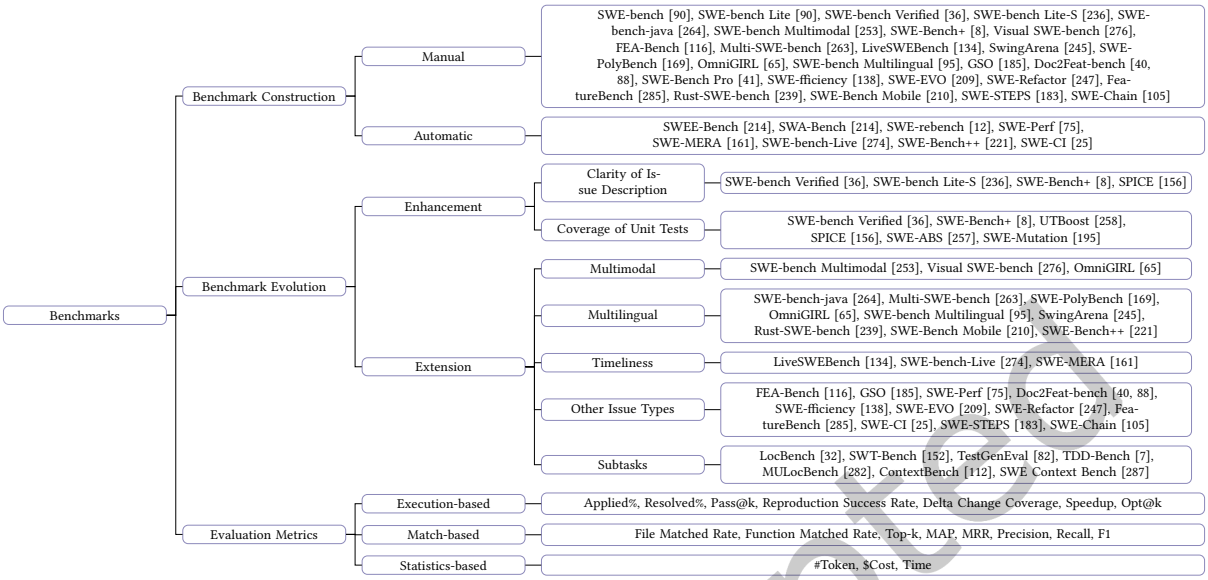


Fig. 1. Taxonomy of issue resolution benchmarks.

addition, performance optimization, and refactoring. Section 4.2 further details the purpose of each benchmark along these evolution directions, and Section 4.4 provides guidance on selecting appropriate benchmarks.

4.1.1 Manually Constructed Benchmarks. Manual construction typically follows the SWE-bench paradigm [90]: researchers collect and filter repository pull requests, configure execution environments, and validate instances through test outcomes before and after patching. Optional human annotation identifies a high-quality subset based on issue clarity and test coverage.

4.1.2 Automatically Constructed Benchmarks. Automatic construction aims to collect large-scale, up-to-date data, primarily by automating the labor-intensive environment setup process [214, 274]. Automated pipelines also continuously collect and validate tasks from repositories and continuous-integration histories [12, 25]. Such methods support training-data collection as well (Section 5.2.1.C). Appendix C.4 presents the detailed construction pipeline and representative implementations of both approaches.

4.2 Benchmark Evolution

Based on the evolution direction of existing benchmarks, we discuss the evolution of issue resolution benchmarks along two main routes: enhancement and extension.

4.2.1 Benchmark Enhancement. Benchmark enhancement aims at the process of filtering high-quality subsets or repairing existing benchmarks through manual or automated approaches, focusing on aspects such as the clarity of issue descriptions or the coverage of unit tests, in order to enable reliable evaluation.

A. Clarity of Issue Description

In benchmarks, issue descriptions are typically derived from natural language explanations that developers provide in real-world pull requests. However, because natural language is often ambiguous and vague, LLMs may fail to accurately capture the requirements, which in turn hinders their ability to solve the tasks.

Existing studies have investigated methods for identifying unclear issue descriptions. OpenAI [36] employs 93 experienced software engineers to annotate the clarity of issue descriptions in 1,699 SWE-bench samples and filters instances based on inter-annotator agreement. Xia et al. [236] manually inspect issue descriptions to

Just Accepted

identify misleading or underspecified statements, filtering 252 high-quality tasks from SWE-bench Lite to create SWE-bench Lite-S. Aleithan et al. [8] conduct an empirical study on SWE-bench and find that solutions to 32.67% of tasks are explicitly provided in the issue descriptions. Building on these findings, Oliva et al. [156] propose SPICE, the first automated annotation method, which constructs rationale-informed prompts from OpenAI's annotations and uses prompt engineering to automate clarity labeling.

B. Coverage of Unit Tests

In real-world software repositories, unit tests are often written for specific issues. As a result, the unit tests used to evaluate solution correctness may be overly specified or excessively broad, and in some cases, even unrelated to the underlying problem [36, 156]. This can lead to situations where correct solutions are rejected or incorrect solutions are accepted. Plausible patches accepted by weak test suites can behave differently from ground-truth patches and thus inflate measured resolution rates [230] (see Section 6.1).

Recent work has focused on automating the identification of issues in test case coverage. Specifically, OpenAI [36] conducts manual annotation to identify overly specific test cases and, based on this effort, constructs SWE-bench Verified. Aleithan et al. [8] study the performance of the state-of-the-art software engineering agent SWE-agent on SWE-bench and find that 31.08% of the test cases are excessively broad. Yu et al. [258] employ differential testing to detect discrepancies between ground-truth patches and candidate patches. These discrepancies are then used as context to guide the LLM in automatically enhancing the tests, thereby reinforcing the unit tests in SWE-bench. Oliva et al. [156] leverage Aider to build an autonomous agent capable of exploring software repositories to assess the coverage of test cases automatically. Beyond coverage, SWE-ABS [257] adversarially strengthens test suites to expose inflated success rates, while SWE-Mutation [195] applies mutation analysis to assess the reliability of the test suites.

4.2.2 Benchmark Extension. Benchmark extension aims to broaden the coverage of issue resolution benchmarks across multimodal inputs, multiple programming languages, time spans, issue types, and related subtasks, in order to achieve comprehensive and thorough evaluation. Appendix C.5 details representative benchmarks along these five directions.

A. Multimodal

Issue descriptions presented solely in textual [36, 90, 264] form are insufficient to capture the diverse application scenarios in software engineering. For instance, domains such as front-end development and game development often rely on visual elements, such as images, to express requirements. To broaden the coverage of issue resolution tasks, researchers have extended the current benchmark framework by incorporating multimodal problem descriptions as inputs.

B. Multilingual

Early issue resolution benchmarks almost exclusively focus on tasks from Python repositories [8, 36, 90, 116, 214], which results in insufficient evaluation of large language models across the diverse software ecosystem. To address this limitation, researchers extend issue resolution tasks to multiple programming languages to enable broader evaluation.

C. Timeliness

Static issue resolution benchmarks [36, 65, 90, 95, 169, 253, 263] cover only a limited number of repositories and remain unchanged after their release. As more advanced LLMs are trained, the data in these benchmarks is likely to have already been exposed, leading to risks of overfitting and data contamination. For example, state-of-the-art models can infer buggy file paths from issue descriptions alone [120] (see Section 6.1). To address this problem, researchers construct dynamic, continuously updated benchmarks.

D. Other Issue Types

Just Accepted

SWE-bench and its variants primarily focus on evaluating end-to-end bug-fixing tasks [36, 90, 95, 253, 263]. However, in real-world software repositories, issues also include non-bug-fixing tasks such as efficiency optimization. Therefore, exploring the performance of LLMs on non-bug-fixing tasks is crucial. Beyond bug fixing, extensions evaluate feature addition [116], runtime performance optimization [75], behavior-preserving refactoring [247], and sequences of interdependent changes in long-term software evolution [209].

E. Subtasks

As mentioned in Section 2.1, issue resolution is a multi-stage task. However, most benchmarks focus solely on end-to-end accuracy evaluation and lack a fine-grained assessment of software engineering agentic systems on the subtasks of each stage. To address this challenge, researchers construct independent benchmarks for subtasks such as issue reproduction and issue localization to gain fine-grained insights into these subtasks. Table 5 (see Appendix C.2) presents the statistics of benchmarks that target the subtasks of issue resolution.

Beyond enabling fine-grained assessment, performance on these subtasks is closely tied to end-to-end resolution outcomes. Existing studies have demonstrated that dedicated localization methods help improve the end-to-end resolution rates when their localization results are fed into downstream patch generation pipelines [32, 89, 261]. Additionally, generated reproduction tests are widely used to filter and rerank candidate patches, improving the precision of submitted patches [5, 152, 236]. Nevertheless, stronger subtask performance is helpful but not sufficient for end-to-end success, since correct localization does not guarantee a correct patch, and weak reproduction tests may accept plausible but behaviorally incorrect patches [230]. Subtask benchmarks should therefore be regarded as diagnostic instruments that decompose end-to-end scores and expose stage-level bottlenecks, rather than substitutes for end-to-end evaluation.

4.3 Evaluation Metrics

Evaluation metrics play an important role in both end-to-end issue resolution evaluation and fine-grained stage-specific evaluation, as they quantify the issue resolution capabilities of software engineering agentic systems. Following the taxonomy construction procedure in Section 3.3, two authors extract and normalize the metrics used in surveyed papers, including their names, definitions, and target stages. We retain recurring and widely used metrics for cross-study comparison. Specifically, we identify all metrics used in the surveyed papers, retain those reported in more than 5% of the papers, and group the retained metrics into execution-based, match-based, and statistics-based metrics. We further discuss these three types of metrics in Appendix C.3.

Across the surveyed papers, we observe that evaluation relies almost exclusively on automatic assessment, and that the choice of metrics is highly correlated with the evaluation target. Specifically, end-to-end evaluation is entirely execution-based, and 76.4% of the surveyed papers report Resolved%, indicating that it serves as the most common metric for the issue resolution task. Beyond Resolved%, metric selection is further differentiated along two dimensions. First, it depends on the task stage. The localization stage is evaluated exclusively with match-based metrics. For example, Top-k, MAP, and MRR for techniques that output explicit localization results, and File/Function Matched Rate derived from the final patch for agentic systems that do not expose such intermediate results. The reproduction stage uses Reproduction Success Rate and Delta Change Coverage to measure whether a generated test reproduces the issue and how well it covers the ground-truth fix. Second, it depends on the issue type. While bug fixing and feature addition tasks are assessed with the correctness-oriented Resolved%, performance issues additionally require Speedup and Opt@k, which extend execution-based evaluation from functional correctness to runtime efficiency. Statistics-based metrics are task-agnostic and serve only as complements to effectiveness metrics, reflecting that current evaluation practice prioritizes effectiveness over efficiency. Manual patch assessment is common in classical APR evaluations because passing the available tests does not always indicate semantic correctness [17, 238]. Accordingly, prior APR studies [238] manually inspect each test-passing patch against the developer patch or intended behavior to determine whether it is actually correct, and typically report

Just Accepted

the number of correct patches together with the correct-to-plausible ratio [17, 238]. Issue resolution benchmarks face the same oracle limitation, since issue-specific tests may not identify some behavioral differences [230]. However, the diversity of issue types further complicates manual patch assessment. For example, feature-addition issues may involve about 180 lines of edits on average [40, 88], making it more difficult to manually determine whether such a generated patch fully implements the intended functionality without introducing unintended behavior than bug-fixing tasks [238]. Additionally, the scale of hundreds of instances makes per-patch manual assessment impractical for routine benchmark evaluation. Manual assessment therefore appears mainly in complementary roles, such as benchmark curation [36] and empirical comparisons between generated and developer patches [8, 29].

4.4 Benchmark Validity and Comparability

To clarify what an evaluation score represents, we examine benchmark validity along three dimensions: realism versus control, contamination and timeliness, and comparability across benchmark families. Specifically, benchmarks built from raw PRs are closer to real development settings, but this realism costs control, since issue descriptions may leak their own solutions [36], and overly broad test suites may let plausible but incorrect patches pass [8, 230]. Filtered benchmarks (e.g., SWE-bench Verified [36]) alleviate this problem through human annotation, but they discard the harder, more ambiguous instances that made the benchmark realistic. Thus, we recommend using raw benchmarks comprising real-world, minimally filtered issues for robustness evaluation, and using filtered benchmarks for reliable comparison across models and methods.

Moreover, contamination poses another threat to benchmark validity, because benchmark instances are mainly collected from public repositories, where both the issue and its ground-truth fix remain openly accessible. Specifically, such contamination arises through two main aspects. The first operates at evaluation time. An agent granted internet access or the full commit history of the repository could directly retrieve the known fix instead of reasoning about the issue. To overcome this, existing benchmarks typically execute agents in isolated environments that provide only a pre-fix snapshot of the repository, without access to the internet or the subsequent commit history [90]. The second is training-data contamination, where the fixed instances of a benchmark gradually enter the training corpora of newer models [120]. For example, prior studies [120, 166] show that models can identify buggy file paths from issue descriptions without a repository environment. Such contamination cannot be excluded by isolation and inevitably grows as the benchmark ages, which leads to a trade-off between static and dynamic benchmarks. Static benchmarks support reproducible and controlled comparison, but their fixed instances face growing training-data contamination risk over time. Dynamic benchmarks [134, 161, 274] alleviate this risk by continuously collecting issues created after training cutoffs, but they are hard to sustain, since repeated manual construction and validation are prohibitively costly and automated construction struggles to guarantee instance quality. Thus, we recommend static benchmarks for rigorous method comparison under a stable protocol, and dynamic benchmarks for assessing the genuine capability of the latest models where contamination is the primary concern.

These differences also undermine comparability across benchmark families. Influenced by factors such as programming language and repository difficulty, resolution rates on different families of benchmarks do not support strict absolute comparison [148]. However, the relative rankings of different models remain a useful reference, since robustness across languages and repositories is an essential dimension of models' capability [29, 128]. Section 6.1 reviews in detail the empirical studies that provide systematic evidence for these evaluation concerns.

RQ1 Summary: Manual curation with execution-based validation remains the common practice of benchmark construction, while benchmarks keep evolving toward multilingual, multimodal, dynamic, and subtask-level evaluation, with execution-based Resolved% as the main metric. Since no single benchmark is universally

Just Accepted

valid, benchmarks should first be scoped by the target language, system kind, domain, and issue type, and then selected by evaluation purpose. For example, filtered benchmarks for reliable comparison, dynamic benchmarks for the latest models, and subtask benchmarks for fine-grained diagnosis.

5 Techniques

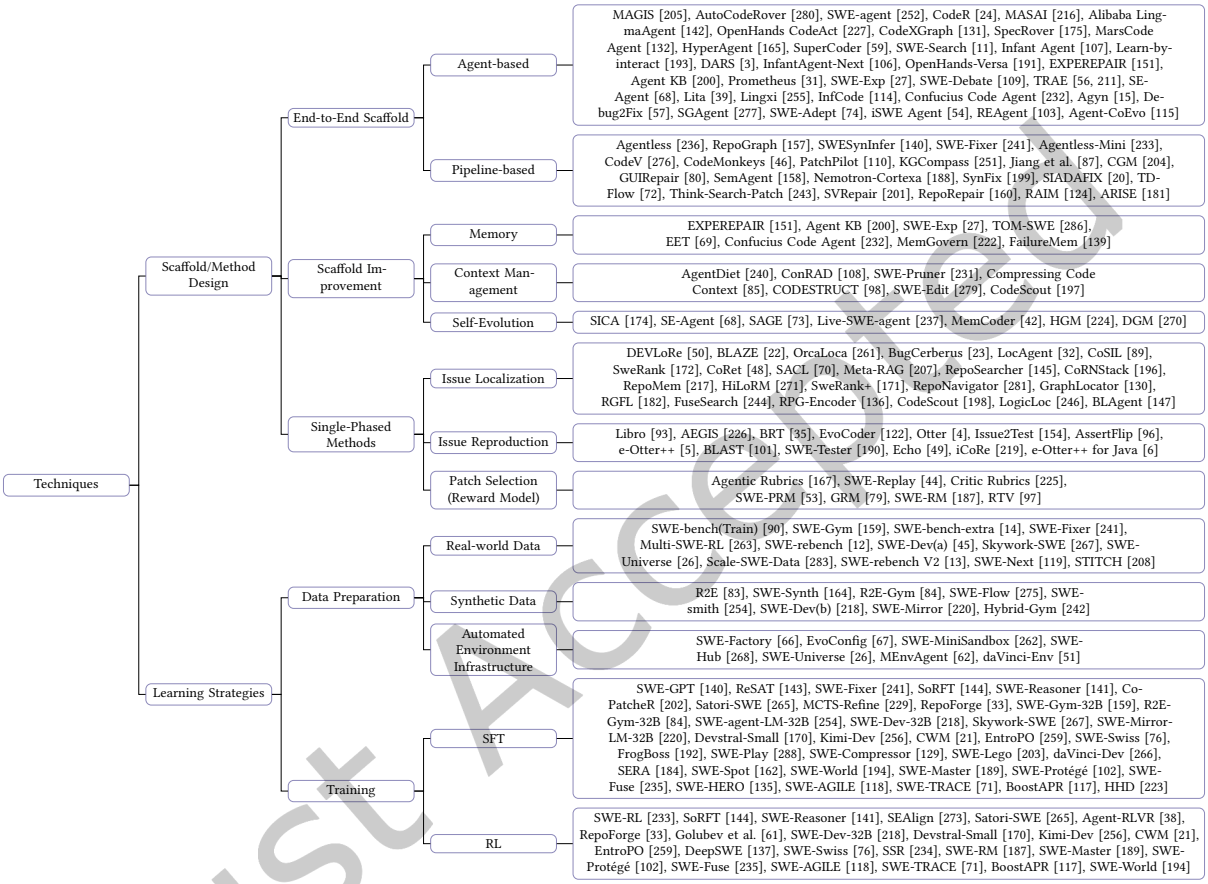


Fig. 2. Taxonomy of issue resolution techniques.

To answer **RQ2**, we present the evolution of issue resolution techniques before and after the paradigm shift along two dimensions, i.e., scaffold/method design and learning strategy, as shown in Figure 2. The two dimensions are distinguished by whether a technique designs the scaffold around a backbone LLM or trains the underlying model itself.

5.1 Scaffold/Method Design

Because the context window of LLMs is limited [2, 63], it is infeasible to feed all code from a repository into an LLM at once to localize focal methods or generate patches. Therefore, researchers design scaffolds for LLMs to solve the issue resolution task or its subtasks. Before February 2025, researchers favored designing complex scaffolds for LLMs to improve performance on issue resolution. According to the scope of the target task, we preliminarily categorize existing scaffolds/methods into end-to-end scaffolds and single-phase methods. In addition, some

Just Accepted

existing works improve existing scaffolds with scaffold-agnostic mechanisms, which we categorize as scaffold improvement.

5.1.1 End-to-End Scaffold. Following the criterion defined in Section 2.1, we categorize end-to-end scaffolds into agent-based and pipeline-based according to whether the control flow is determined by the LLM at inference time or predefined by human designers. Agent-based scaffolds and pipeline-based scaffolds both generally follow the common work phases of Section 2.1, which Appendix A describes in detail. Agent-based scaffolds [15, 39, 54, 57, 74, 103, 114, 115, 277], such as SWE-agent [252] and OpenHands [227], equip one or more agents with tools to autonomously navigate repositories and generate patches, offering high flexibility and autonomy. However, constrained by the planning, decision-making, and instruction-following capabilities of the underlying LLM [89, 273], agent behavior may deviate from the intended pipeline, potentially leading to task failure or system crashes. Pipeline-based scaffolds [20, 72, 124, 160, 181, 201, 243], such as Agentless [236] and PatchPilot [110], instead execute human-designed, fixed phases for localization, repair, validation, and patch selection sequentially. Each phase is explicitly controlled, resulting in more stable and reliable execution [110] but lower autonomy. For scaffolds that combine both designs, such as pipelines whose individual phases internally contain bounded feedback loops, we classify them by their global control flow.

Table 7 (see Appendix D.1) presents the chronological development of existing end-to-end scaffolds, highlighting their designs across the phases of the pipeline described in Appendix A, as well as their unique characteristics and performance-cost trade-off. Since the performance score and cost are self-reported, we draw quantitative comparisons only between methods evaluated on the same benchmark with the same backbone. Under such matched settings, pipeline-based methods deliver comparable effectiveness at lower and more predictable cost. For example, Agentless [236] resolves more instances than the agent-based MASAI [216] at roughly one-third of the cost under the same setting. This cost difference arises because a pipeline bounds the number of LLM calls in each phase, whereas an agent trajectory accumulates context over multiple rounds of tool interactions, making its cost higher and more variable. Beyond such matched pairs, the highest reported resolved rates come from agent-based scaffolds [151, 232, 255]. However, these systems also adopt the strongest and most recent backbones, so scaffold type and backbone capability are confounded in the reported numbers, and we therefore do not attribute the effectiveness gap to the scaffold design alone. In practice, pipeline-based methods are preferable under tight budgets or with weaker backbones, where the predefined workflow compensates for limited planning ability, whereas agent-based methods are preferable when frontier backbones are available and maximizing the resolution rate justifies the higher and less predictable per-issue cost. In the following, we categorize and analyze existing methods with respect to the key design dimensions of each phase in the issue resolution pipeline. We do not distinguish the "Repair" phase in detail, as all scaffolds share the same action at this step: editing the codebase. They differ only in engineering implementation, which is beyond the scope of this survey.

A. Repository Representation (Repo Preprocessing Phase)

Constrained by the semantic gap between natural language requirements and source code, LLMs face challenges in capturing the complex dependencies and semantic information within repositories. To better understand repository structures and code semantics, some approaches generate structured representations of the codebase during the repo preprocessing phase, thereby bridging the gap between natural language and code. According to the relations they preserve, existing representations can be divided into two types: *tree-based representations*, which organize repository files, classes, and functions into containment hierarchies, and *code graphs*, which additionally encode relations such as function calls, inheritance, imports, references, and data flows (detailed in Appendix D.2).

The two types trade construction cost against the relations they expose. Trees are cheap and require no execution, but preserve only containment relations, leaving cross-file dependencies such as calls and imports invisible. Code graphs recover these dependencies at the cost of additional construction and retrieval mechanisms [133],

Just Accepted

and their coverage is bounded by static analysis, so dynamic behavior remains outside the graph. Code graphs have themselves evolved along two axes, from single-relation call graphs toward multi-source graphs that also absorb documentation [31, 132], and from graphs consumed as flat context toward graphs exposed as queryable interfaces [31, 131]. Trees therefore remain the default in pipeline-based scaffolds, where the representation must fit the context window at negligible cost, whereas graphs are adopted by systems that can afford a dedicated retrieval mechanism to exploit them.

B. Localization (Localization Phase)

Accurate localization serves as a critical basis for effective issue resolution [149]. Its objective is to identify a set of potentially faulty code snippets within a specified scope of repository files. According to the evidence they use to link an issue to code, existing localization methods can be divided into five types: *BM25* by lexical overlap, *spectrum-based methods* by execution coverage, *embedding-based methods* by semantic similarity, *navigation-based methods* by LLM-driven repository exploration, and *graph-based methods* by structural relations over the repository graphs of Appendix D.2 (detailed in Appendix D.3).

The five types primarily differ in the cost of the evidence they rely on and the failure modes that this evidence introduces. BM25 is the cheapest signal, requiring neither training, execution, nor model calls, but it is vulnerable to lexical mismatch. Embedding retrieval alleviates this mismatch, yet chunking disrupts cross-file context, and general-purpose encoders poorly capture the asymmetry between issue descriptions and code. Spectrum-based localization adds execution evidence and complements text-based methods, but it depends on a trigger test, which is often absent in the issue resolution task, and therefore remains mainly auxiliary. Navigation trades search coverage for cost predictability. Open-ended exploration can recover from early mistakes but may fail to converge, whereas staged cascades cannot revisit pruned files. Graph-based methods capture structural dependencies, but their recall is ultimately bounded by omissions in the underlying static-analysis graph.

C. Reproduction Test Generation (Patch Validation Phase)

Reproduction tests help localize an issue, filter candidate patches, and check whether a patch satisfies the reported behavior, but they are often absent from issue descriptions. Scaffolds therefore either prompt an LLM directly to generate tests [46, 80, 110, 158, 175, 188, 233, 236] or let an agent inspect the repository before generation [3, 24, 31, 56, 132, 151, 191, 199, 200, 216, 227, 251, 252]. For example, Agentless [236] prompts the LLM for a self-contained script reproducing the reported behavior and keeps it only if it fails on the unpatched repository, so that a candidate patch is accepted only when the same script later passes. This pre-patch check makes the mechanism usable without ground truth, but it verifies only that the script fails, not that it fails for the reported reason. Agentic generation instead inspects fixtures and helpers before writing the test, raising the chance of a repository-consistent test at the cost of additional interaction rounds. Dedicated reproduction methods are discussed in Section 5.1.3.B.

D. Regression Test Selection (Patch Validation Phase)

Regression test selection checks whether candidate patches break existing behavior by selecting tests related to the modified code. Current scaffolds rely mainly on LLM-based selection [56, 110, 175, 191, 199, 200, 227, 233, 236], which may discard correct patches when irrelevant tests are selected or miss regressions when relevant tests are omitted. For example, Agentless [236] records which tests pass before patching, asks the LLM which of them relate to the edited code, and rejects any candidate patch that turns a selected passing test into a failing one. The mechanism is unsound in both directions because the selection step is a model judgment rather than a dependency analysis. Although program-analysis techniques exist [94], they have not been integrated into these scaffolds, and available Python techniques remain unsafe [18, 94]. This category has thus seen the least design evolution of the whole validation phase, as scaffolds have converged on the same LLM-based selection rather than importing the analysis-based selection developed by the testing community. Trajectory-level failure analyses reinforce the importance of the patch validation phase, showing that many unresolved instances stem from frequent runtime errors surfaced during test execution and from stage-specific mistakes [30, 128] (see Section 6.2).

Just Accepted

E. Patch Rerank (Patch Selection Phase)

After test-based filtering, scaffolds commonly rerank plausible patches through self-consistency-based majority voting [46, 56, 80, 132, 157, 188, 233, 236, 251, 276] or LLM-as-a-Judge assessment [3, 46, 56, 110, 151, 158, 175, 216]. For example, Agentless [236] normalizes the sampled patches so that formatting differences do not create spurious variants, and submits a representative of the largest equivalence class. Voting needs no additional model capability but is blind to the content of the patch and rewards whatever the backbone is biased toward, whereas LLM-as-a-Judge inspects the semantics of each candidate against the issue but inherits the judging model's own limits. These strategies can be combined: TRAE applies code review before voting [56, 211], while KGCompass performs rule-guided review before selection [251]. Such hybrids mark the evolution of the category from a single scoring rule toward staged selection that filters semantically before counting. However, existing empirical studies (see Section 6.1) show that patches passing all validation tests can still deviate behaviorally from ground-truth patches due to limited test coverage [230], and may fall short of developer patches in code quality and requirement compliance [29]. Thus, test-based rerank selects the most promising candidate rather than guaranteeing correctness. Methods dedicated to patch selection will be discussed in Section 5.1.3.C.

5.1.2 Scaffold Improvement. Besides building new scaffolds, complementary works develop scaffold-agnostic mechanisms that improve existing scaffolds. According to the target of improvement, we categorize these mechanisms into three types: memory, which accumulates reusable knowledge across issues, context management, which steers the content fed into the context window, and self-evolution, which lets the agentic system refine its own design.

A. Memory

Since most scaffolds resolve each issue in isolation and discard the trajectory afterwards, they ignore the knowledge and lessons that can be learned in previous exploration. Memory mechanisms equip a scaffold with a long-term store that carries such knowledge and lessons across issues. According to the source of the stored knowledge, existing memory mechanisms work in two ways: distilling experience from the system's own trajectories, and importing knowledge from sources beyond the system. Specifically, trajectory-based methods summarize previous resolution processes into structured entries and retrieve relevant ones to guide new resolutions [27, 69, 139, 151, 200]. For example, EXPEREPAIR [151] stores concrete demonstrations as episodic memory and summarized repair insights as semantic memory, and retrieves both to guide a new issue, while SWE-Exp [27] extracts transferable experience from both successful and failed trajectories. External-source methods instead structure knowledge held by humans, such as developer experience on GitHub [222], user intents [286], and notes accumulated across sessions [232]. For example, MemGovern [222] transforms unstructured GitHub discussions into governed experience cards and matches them to agents through logic-driven retrieval. The trade-off separating the two is provenance: self-distilled memory is automatic and repository-specific but can entrench the system's own mistakes, whereas external memory carries human-validated knowledge but requires curation and may not match the current codebase.

B. Context Management

Since agent trajectories accumulate context over multiple rounds of tool interactions, as discussed in Section 5.1.1, the accumulated context inflates cost and dilutes the attention of the LLM [125]. Context management mechanisms control what enters the context window of the LLM during resolution. According to the direction of adjustment, existing methods work in two ways: compressing low-value content and enriching informative content. Specifically, compression methods shrink the context either by removing content that no longer contributes to resolution [85, 231, 240] or by reorganizing the interaction interface so that the same information occupies less context [98, 279]. For example, AgentDiet [240] identifies useless, redundant, and expired information in the trajectory and removes it during execution, and CODESTRUCT [98] lets agents read and edit abstract syntax tree entities instead of matching raw text. Enrichment methods instead improve the quality of the supplied

Just Accepted

context [108, 197]. For example, CodeScout [197] pre-explores the codebase to rewrite underspecified issue descriptions into actionable problem statements. The two directions trade against each other, since compression lowers cost but risks discarding evidence the agent later needs, whereas enrichment improves grounding at the price of additional pre-processing.

C. Self-Evolution

Since handcrafted scaffolds are fixed at development time, adapting them to new backbones or task distributions requires costly manual redesign [237]. Self-evolution mechanisms enable the agentic system to optimize its own scaffold or resolution strategy based on execution feedback. According to the object of evolution, existing methods work at two levels: evolving the scaffold implementation and evolving the resolution strategy. Specifically, implementation-level methods let the agent modify its own code and retain variants that are validated by benchmark performance [174, 224, 237, 270]. For example, the Darwin Gödel Machine [270] maintains an archive of agent variants and grows it through validated self-modifications, and Live-SWE-agent [237] evolves a basic scaffold with only bash tools on the fly while solving real issues. Strategy-level methods instead keep the scaffold fixed and refine how it resolves issues, such as revising and recombining prior trajectories [68], abstracting plans from grounded experience [73], and accumulating project knowledge from commit history [42]. For example, SE-Agent [68] revises, recombines, and refines prior trajectories to explore the solution space more effectively. Implementation-level evolution can reach designs a human would not write but needs a trusted validation signal, since the archive is only as meaningful as the benchmark that admits variants into it, whereas strategy-level evolution is cheaper and safer but cannot escape the limits of the fixed scaffold.

5.1.3 Single-Phased Methods. Current single-phased methods focus on three subtasks: issue localization, issue reproduction, and patch selection. Issue localization requires the agentic system to identify the code locations that need modification, serving as the foundation for subsequent editing. Issue reproduction aims to generate a test case for the target issue that fails in the buggy repository but passes in the fixed one, thereby assisting in both localization and verification of issue resolution. Patch selection aims to identify, among multiple candidate patches, the one most likely to resolve the issue, determining the quality of the final submission. Since these subtasks are critical steps in issue resolution, improvements on them have been shown to translate into higher end-to-end resolution rates [32, 89, 152, 261].

A. Issue Localization

Different from the localization component of the scaffold discussed in Section 5.1.1.B, we discuss methods dedicated to issue localization, whose results can be leveraged to enhance the repair phase [50, 130, 136, 147, 171, 182, 198, 244, 246, 271, 281]. These methods split by whether localization capability is placed in the inference-time procedure or in the model parameters.

A.1 RAG-based Methods. Retrieval-augmented generation localizes issues through vector, graph, or navigation-based repository retrieval. Vector RAG ranks code segments by embedding similarity and is discussed with trained embedding models in Section 5.1.3.A.2. Graph RAG performs multi-hop structural retrieval. For example, LocAgent [32] parses the repository into a heterogeneous graph of files, classes, and functions and equips an agent with traversal tools, so that it can hop from an entity named in the issue to its callers before ranking suspicious locations. Navigation-based RAG instead lets agents inspect repositories dynamically. It improves search through action scheduling and context pruning [261], multi-agent codebase compression [207], or related guided navigation designs [32, 217]. Hybrid approaches combine these signals, as SACL does with vector retrieval and agent navigation [70]. The shared advantage is that these methods need no training and transfer to any repository, so they follow a frontier backbone as it improves. The shared cost is that every issue pays for repeated retrieval and reasoning at inference time.

A.2 Fine-Tuning-based Methods. Fine-tuning-based localization trains either generation models to predict locations or embedding models to retrieve them. For generation models, it learns defect patterns from segmented code

Just Accepted

or intermediate program representations [22, 23]. For embedding models, it improves retrieval through multilingual contrastive data [196], real issue–modification pairs [172], or repository structure and call dependencies [48]. For example, SweRank [172] contrastively trains a retriever on issue-modification pairs mined from commit history and pairs it with a trained reranker, turning localization into a retrieve-then-rerank pass. Compared to RAG-based methods, this moves cost from inference to training, giving cheap and reusable localization, but the learned signal reflects the repositories and issue styles seen in training and must be refreshed as projects drift.

B. Issue Reproduction

Automatically reproducing issues from issue descriptions enables timely localization and fixing, thereby improving development efficiency. Existing approaches have focused on narrow bug categories including Android issues [81] and configuration-induced faults [52], but these constitute only a small fraction of real-world defects.

Given the strong understanding and generation capabilities of LLMs, some works use basic prompting to generate reproduction tests. For example, AssertFlip [96] first generates tests that pass on the buggy version and then flips assertions to obtain failing reproduction tests. Building on basic prompting, several approaches introduce systematic multi-turn refinement. For example, AEGIS [226] employs a finite-state-machine-based optimization module to orchestrate multi-turn interactions and script adjustments for controlled refinement. Another line of work enhances LLM-based generation with repository inspection and rule-based reasoning. Otter [4] augments LLM outputs with rule-based analysis and a self-reflection planning stage to support test-driven development. More recent methods further improve reproduction through graph-enhanced retrieval and execution feedback [49, 219], continuous learning for defective-code reproduction [122], dedicated training for issue reproduction [190], and extension to additional languages [6].

C. Patch Selection

Dedicated patch selection methods build verifiers that assess candidate patches or their resolution trajectories and select the most promising one for submission. They follow two designs: scoring-based methods independently rate candidates with repository-derived rubrics, outcome-trained critics, or process signals [53, 79, 167, 187, 225], whereas comparison-based methods rank candidates through rollout tournaments or branch from promising archived steps [44, 97]. SWE-RM exemplifies execution-free fine-grained scoring [187], while RTV compares structured rollout summaries in recursive small-group tournaments [97], so that the verifier only ever performs local comparisons instead of scoring the whole pool at once. Scoring is cheap and parallel but requires the verifier to be calibrated on an absolute scale, whereas comparison avoids calibration by asking only which of two candidates is better, at the cost of more verifier calls. Unlike the in-scaffold rerank of Section 5.1.1.E, these verifiers are trained artifacts, which is what allows them to be reused as reward models. Scoring verifiers can also supply rewards for reinforcement learning [79, 187], as discussed in Section 5.2.2.B.

5.2 Learning Strategies

After the release of DeepSeek-R1 [63], agentic reinforcement learning has become an increasingly important strategy for training issue-resolution agentic systems. As shown in Table 9, reinforcement learning is used by none of the four representative models released before February 2025, but by 24 of the 39 released thereafter, accounting for 61.5%. Moreover, the highest reported SWE-bench Verified performance in Table 9 is achieved by an RL-based model. These results indicate a recent shift toward reinforcement-learning-based training in issue resolution.

5.2.1 Data Preparation. The process of preparing real-world training datasets for training issue resolution domain-specific models is similar to the benchmark construction process described in Section 4.1. Given the scarcity of annotated data and the gradual depletion of real issue resolution samples, researchers have also explored synthetic data generation methods based on real repositories to expand the scale of training data. Table 8

Just Accepted

(see Appendix D.4) summarizes the existing real-world and synthetic datasets. Since both directions depend on executable environments, recent work further builds automated infrastructure for environment construction.

A. Real-world Data

Real-world datasets are constructed from open-source repositories and issue–commit pairs. SWE-bench (Train) [90] is the first large-scale dataset to organize 19k real issue-fix pairs, providing training samples that reflect real-world distributions. However, the lack of executable environments prevents models trained on these datasets from obtaining stable reward signals, limiting their use to supervised fine-tuning. With the growing success of RL, recent models such as DeepSeek-R1 [63] show that rule-based rewards can be effective, making executable environments a key requirement for new datasets. Execution-backed datasets are built either by curating an environment per instance [45, 159] or by automated pipelines that continuously harvest and validate new tasks [12, 13, 119, 208, 263, 267, 283].

B. Synthetic Data

Since curating real instances and their executable environments still demands hundreds of hours of manual effort and often suffers low success rates [220, 254], synthetic data instead generates instances and environments from existing repositories. According to where the defect of a synthesized instance originates, existing methods work in two ways: deriving tasks from real code changes and injecting faults into prepared environments. Specifically, PR-derived methods recover a task from a change that developers actually made, reverse-translating the commit into a problem statement and generating the tests that witness it, which removes the reliance on human-written issue descriptions and unit tests while staying within the distribution of real development [83, 84, 218, 275]. For example, SWE-Flow [275] recovers an incremental test-driven development schedule from the unit tests of an existing repository. Bug-injection methods instead start from a repository whose environment is already configured and introduce the fault themselves, either mutating working code [164, 254] or transferring bug-fix logic abstracted from other projects [220]. For example, SWE-Synth [164] has LLM agents inject bugs into a working repository and then simulate the human debugging process over them to obtain verifiable instances. Injection is the cheaper of the two because one configured environment amortizes over many generated instances, but its defects are defined by the mutation applied rather than by a behavior a user reported, so it drifts further from real issue distributions. Therefore, recent work mixes synthetic with real tasks rather than replacing them [242].

C. Automated Environment Infrastructure

Because both real-world collection and data synthesis rely on executable environments, whose construction dominates the manual effort and overhead of dataset production [220, 254], automated environment infrastructure is proposed to construct and manage executable environments without manual configuration. Existing works can be divided into two aspects: automating environment setup and environment execution cost reduction. Specifically, automated setup methods employ agentic pipelines that generate environment configurations, iteratively diagnose and repair build failures, and validate the results through executed tests [26, 51, 62, 66, 67, 268]. For example, SWE-Factory [66] builds environments with a multi-agent system and validates instances through exit-code-based log parsing and fail-to-pass checks. Cost reduction methods aim to reduce the runtime cost of using massive environments. For example, SWE-MiniSandbox [262] replaces per-task containers with kernel-level isolation and environment pre-caching. The two aspects address the two costs of environments respectively, namely the one-off cost of making a task runnable and the recurring cost of running it once per rollout, and the emergence of the latter reflects that reinforcement learning executes environments continuously rather than once.

5.2.2 Training. The training of domain-specific models for the issue resolution task has evolved rapidly alongside the expansion of SWE-bench-style datasets and the emergence of open scaffolds such as OpenHands [227], Agentless [236], and SWE-agent [252]. Table 9 (see Appendix D.5) summarizes representative models, their training paradigms, and their performance on SWE-bench Verified.

Just Accepted

A. Supervised Fine-Tuning

Supervised fine-tuning (SFT) is a fundamental approach for training domain-specific models by injecting task knowledge through input–output pairs. In issue resolution, common SFT strategies include process-oriented methods and teacher-model distillation.

Process-oriented approaches decompose issue resolution into pipeline stages such as localization and repair, and supervise each stage separately. Existing methods model these intermediate processes by simulating developer interactions and tool use [140], enriching stage-level supervision with repository structure or specialized retrieval and editing modules [143, 202, 241], or training deeper reasoning trajectories with rejection-based refinement [141, 229]. Thus, they shift supervision from static issue–patch pairs toward the structure and reasoning of the resolution process.

Teacher-model distillation methods run a powerful model (e.g., Claude-4.5 [9], GPT-4o [2]) within a scaffold to sample complete resolution trajectories, retain the trajectories whose final patches pass validation, and fine-tune a smaller open model on them, so that the student model imitates the teacher's multi-step reasoning and tool usage. Models released with new datasets commonly use this strategy to validate dataset effectiveness [84, 159, 220, 254, 267]. Distillation also supplies cold-start policies before RL [71, 117, 194, 259], while other open models rely predominantly on SFT over curated trajectories [129, 192, 288]. The two strategies differ in where supervision comes from and what it costs. Process-oriented supervision is derivable from commit history alone and therefore scales with text-only corpora, but its golden intermediate outputs are reconstructed from the final fix rather than observed, so they describe a path the developer never took. Distillation instead observes genuine trajectories, yet it is bounded by the teacher, requires executable environments to filter trajectories by validation outcome, and incurs the teacher's inference cost. Because filtering already presumes an execution oracle, distillation naturally became the cold start for the reinforcement learning that follows.

B. Reinforcement Learning

Reinforcement learning (RL) has demonstrated a distinctive ability to cultivate long-horizon decision-making behaviors that static supervised methods fail to capture [63, 137]. Recent open-source efforts have started exploring this complexity through RL-based training frameworks. During RL training, the model rolls out resolution trajectories by interacting with a repository environment, receives a reward that scores each trajectory, and updates its parameters with policy optimization algorithms such as PPO [179] to make high-reward behaviors more likely. Unlike SFT, which imitates fixed demonstrations, RL lets the model learn from feedback on its own explored trajectories. According to the type of reward model used, RL training methods can be categorized into process reward models (PRMs) and outcome reward models (ORMs).

PRM-based reinforcement learning rewards intermediate states or actions to teach effective resolution procedures rather than only successful outcomes. For example, SoRFT [144] rewards file, function, and line localization and code editing separately against the developer's fix, so that credit is assigned to the stage that produced the error rather than to the trajectory as a whole. PRMs provide fine-grained guidance but require reliable process-level signals. This is the binding constraint on the category, since intermediate correctness has no execution oracle and must itself be approximated by rules or a learned critic, which is why PRM adoption remains far behind ORM despite its finer credit assignment.

ORM-based reinforcement learning instead uses verifiable final outcomes. Existing methods derive rewards from commit histories or candidate checks [141, 233], use unit-test outcomes directly [38, 76, 170, 218, 256], or combine outcome verification with structured exploration and policy optimization [61, 265, 273]. In the canonical workflow, the patch produced by a rollout is executed against the fail-to-pass and pass-to-pass tests, and the binary outcome becomes the reward for the whole trajectory, so supervision requires no human annotation but arrives only at the end of a long horizon. ORMs scale because outcomes are automatically verifiable, but their supervision is only as reliable as the underlying oracle. Nevertheless, the reliability of outcome rewards is affected by the test suites that produce them. Since weak test suites may accept plausible patches that are behaviorally

Just Accepted

incorrect [29, 230], purely outcome-driven rewards risk reinforcing such patches during training. Efficiency analyses further reveal token snowballing and costly failure patterns in current systems [47], suggesting that reward designs should also account for the cost of the resolution process. We further discuss this in Section 6.1.

RQ2 Summary: Existing techniques advance along two complementary dimensions: scaffold design and model training. Among scaffolds, pipeline-based designs suit tight budgets and weaker backbones, agent-based designs suit frontier backbones, and scaffold-agnostic mechanisms for memory, context management, and self-evolution improve both without redesign. Training has shifted decisively toward agentic reinforcement learning, but its ORM remains bounded by test suite quality. Thus, PRM may provide a promising direction.

6 Empirical Studies

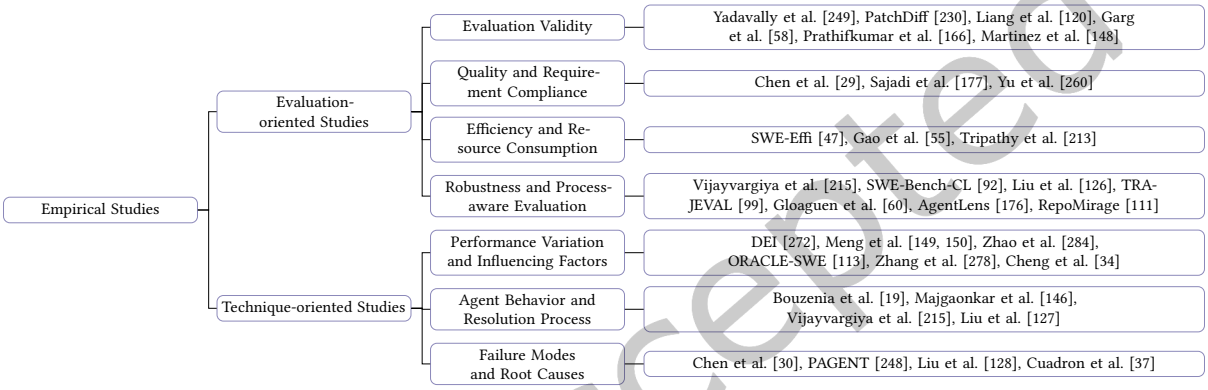


Fig. 3. Taxonomy of empirical studies of issue resolution.

To answer **RQ3**, this section synthesizes what empirical studies establish about the reliability of reported resolution rates and about the causes of the performance differences observed among agentic systems. Figure 3 presents the resulting taxonomy of empirical studies constructed through the open coding procedure in Section 3.3. Its top level separates evaluation-oriented studies, which scrutinize what a reported resolution rate does and does not certify (Section 6.1), from technique-oriented studies, which trace performance differences to the information, control, and process characteristics of individual agentic systems (Section 6.2). We then analyze the findings of each group and identify what is still missing before these systems can be used in real projects.

6.1 Evaluation-oriented Studies

Evaluation-oriented studies show that a reported resolution rate is a useful signal under a fixed protocol, but it is not sufficient evidence of practical issue-resolution capability. They expose validity threats, assess properties hidden by outcome metrics, and test whether conclusions persist beyond standard benchmark settings.

Evaluation Validity. To determine whether benchmarks, tests, and evaluation protocols faithfully reflect the capability of an agentic system, evaluation-validity studies examine three distinct threats to reported resolution rates. First, weak tests can accept patches that differ behaviorally from ground-truth patches, as shown by PatchDiff [230]. Second, model memory and contamination can inflate resolution rates on SWE-bench and SWE-bench Verified [120, 166]. Third, benchmark prompts and leaderboard settings can obscure the gap between formal tasks and realistic developer requests [58, 148]. Execution-free evaluation mechanisms provide a complementary signal for judging repository-level changes without relying on a prescribed test outcome [249]. Together, these studies show that reported resolution rates support controlled comparison, but do not alone establish trustworthy real-world capability.

Just Accepted

Quality and Requirement Compliance. To assess whether test-passing patches meet the broader requirements of real projects, quality and requirement-compliance studies examine evidence beyond functional correctness. Comparisons with developer patches reveal differences in code quality and requirement compliance [29], security analyses expose vulnerabilities introduced by generated fixes [177], and design-constraint evaluation identifies violations that conventional tests may miss [260]. Behavioral correctness, maintainability, security, and project constraints are therefore complementary requirements rather than interchangeable metrics that a high resolution rate can compensate for.

Efficiency and Resource Consumption. To assess whether an agentic system is practically useful under resource constraints, efficiency studies relate successful resolution to the resources required to obtain a successful patch. SWE-Effi identifies token snowballing and costly failure patterns [47], while turn-control strategies, energy consumption, runtime, and memory use reveal further trade-offs among agentic systems [55, 213]. These findings distinguish raw effectiveness from deployable effectiveness, since a modest gain in resolution rate may not justify substantially greater resource consumption.

Robustness and Process-aware Evaluation. To assess whether agentic systems remain reliable when conditions depart from standard benchmarks, robustness and process-aware studies examine interactive requirements, continual learning, trajectory quality, and perturbed repository contexts. Interactive tasks show that agentic systems struggle with ambiguous requirements but can improve by seeking clarification [215]. SWE-Bench-CL examines continual learning, whereas process analyses and trajectory-level evaluation identify properties that outcome metrics conceal [92, 99, 126, 176]. Repository perturbations reveal whether agentic systems reason over context rather than exploit spurious cues [111]. Most evidence remains centered on fixed SWE-bench-style protocols, which enable controlled comparison but bound the generalizability of their conclusions. Well-timed actions and reliable trajectories are therefore necessary for trusting a reported success [60].

6.2 Technique-oriented Studies

Technique-oriented studies explain performance variation through the information available to an agentic system, the control mechanisms that govern its actions, and the reliability of its resolution process.

Performance Variation and Influencing Factors. To explain why agentic systems achieve different outcomes on the same task, performance-variation studies examine the information and control available to them. Different agentic systems solve complementary subsets of tasks [272], and performance differences across leading agentic systems depend on the instances they can or cannot resolve [149, 150]. Localization quality, repository context, search budget, and oracle signals such as tests, locations, and APIs determine how much useful information reaches the agentic system [113, 284]. Cogeneration experiments further show that coupling bug reproduction tests with fixes influences how an agentic system behaves while preserving the generation rate of plausible fixes, indicating that workflow structure is another performance factor [34]. Rules, guidance, and guardrails further shape whether the agentic system can use that information reliably [278]. Thus, performance cannot be attributed to the backbone model alone. It reflects the joint design of information access and control.

Agent Behavior and Resolution Process. To identify process-level determinants of success and failure, behavior studies analyze how agentic systems interpret requirements, plan, and act during resolution. Performance variation is also process-level rather than solely model-level. Agentic systems must recognize ambiguity and seek clarification when requirements are underspecified [215]. Successful and failed trajectories differ in action sequences, iteration patterns, and semantic consistency [19, 146]. Agentic systems do not always execute their own plans faithfully [127]. These findings shift the explanation of failure from an undifferentiated lack of model ability to observable weaknesses in interaction and trajectory discipline.

Failure Modes and Root Causes. To locate recurrent breakdowns that the design of an agentic system can target, failure-mode studies diagnose where and why resolution processes fail. Such failures are recurrent and diagnosable

Just Accepted

rather than random residual errors. Large-scale trajectory and test-log analyses identify frequent runtime errors and process-level mistakes [30]. Failed-patch categories can support targeted repair [248], and stage-based taxonomies expose where unresolved instances break down [128]. Safeguards for error-prone mutating actions provide a concrete example of using such diagnoses to improve the reliability of agentic systems [37]. These studies motivate phase-level logging, diagnostic evaluation, and targeted safeguards instead of relying exclusively on end-to-end resolution rates.

RQ3 Summary: Reported resolution rates support controlled comparison but do not certify practical capability, since weak tests [230], contamination [120, 166], unrealistic task framing [58, 148], and unreliable trajectories [60, 176] can make a resolved instance overstate real ability. Performance variation is driven more by information access, control design, and recurrent process-level failures than by the backbone model or the aggregate rank of an agentic system. In practice, assessment requires the resolution rate to be reported alongside patch quality, security, robustness, and resource evidence, supported by phase-level and trajectory-level diagnostics. These conclusions calibrate the benchmark claims in Section 4 and the technique claims in Section 5, while motivating the multi-perspective and fine-grained evaluation agenda in Section 7.

7 Research Opportunities and Road Ahead

Despite substantial progress by large language model-based agentic systems on the issue resolution task, achieving fully autonomous and reliable automated software maintenance remains highly challenging. To answer **RQ4**, this section outlines the current challenges extracted from previous sections and surveyed papers and identifies research opportunities and future directions on the path toward automated issue resolution. Specifically, during the open coding procedure in Section 3.3, we additionally record the limitations and future work explicitly stated in the coded papers, cross-check them against the cross-study insights in Section 6, and identify unique challenges according to previous parts of this survey. We present the challenges in the organizational order of this survey, moving from evaluation to techniques and then to emerging directions, so that each challenge follows the sections whose evidence it draws on. This order is presentational and does not itself encode priority. To indicate where the field places its own emphasis, we rank the challenges by community attention. Specifically, we report for each challenge below the number of surveyed papers that explicitly raise it as a limitation or future work during the open coding described above. By this measure, the community concentrates most on training domain-specific models (65 of 242 papers, 26.9%), repository knowledge representation (60, 24.8%), and validation via tests (49, 20.2%). Although the two evaluation-related challenges are discussed less frequently than these three (in 37 and 25 papers, respectively), the empirical evidence presented in Section 6 indicates that unreliable evaluation makes it difficult to assess progress toward addressing the remaining challenges. We therefore regard this mismatch between limited attention and foundational importance as an important finding in its own right.

7.1 Evaluation of Agentic Systems

Multi-Perspective Evaluation. This challenge is distilled from the metric analysis in Section 4.3 and the evaluation-oriented studies in Section 6.1, which show that resolution rate alone omits deployment-critical properties such as robustness, security, code quality and maintainability, design/architectural-constraint compliance, and performance [29, 128, 177, 215, 260]. In total, 37 of the 242 surveyed papers (15.3%) explicitly raise this challenge. A pragmatic route is to improve existing issue benchmarks by associating them with property-specific checkers, such as perturbed issue descriptions for robustness, static security scanning of generated patches, static code-quality and maintainability metrics (e.g., complexity, duplication, code smells) benchmarked against the pre-patch baseline, automated checks of project-specific design/architectural constraints, and performance profiling of patched code, rather than building new benchmarks from scratch. Such benchmarks speak primarily to industrial

Just Accepted

adopters deciding whether a system is fit for deployment, whereas designing the underlying metrics and testbeds remains a task for academic researchers.

Fine-Grained Evaluation. This challenge follows from the end-to-end benchmark design in Section 4 and the trajectory and failure analyses in Section 6.2, which show that aggregate resolution rates conceal stage-specific failures [19, 30, 128]. In total, 25 of the 242 surveyed papers (10.3%) explicitly raise this challenge. Concretely, such frameworks should measure phase-level capabilities along the workflow in Section 5, including localization accuracy at the file and function levels, the fault-revealing quality of reproduction tests, and patch correctness beyond passing the available test suite. Beyond final task success, trajectory-level process quality and efficiency, including successful tool execution, redundant actions or interactions, and token consumption, deserve comparable attention. Such evaluation can be made more rigorous and informative when benchmarks provide phase-level reference information, such as ground-truth fault locations, and when agent scaffolds record execution trajectories using a standardized format. These diagnostic signals matter most to researchers and scaffold developers iterating on system designs.

7.2 Repository Knowledge Representation

The repo-preprocessing designs in Section 5.1.1 predominantly represent code as natural language, sometimes augmented with trees or entity-relation graphs [110, 236, 241]. In total, 60 of the 242 surveyed papers (24.8%) explicitly raise this challenge. Future representations should more directly encode program semantics and execution behavior while integrating repository artifacts beyond code, including documentation, images, and demonstration videos. Research should determine which representations improve localization and architectural understanding, how they remain synchronized after edits, and whether their benefits justify construction and retrieval costs. Progress on both fronts rests largely with the repository representation research community, whose advances can then be adopted by builders of agentic systems.

7.3 Validation via Tests

This challenge arises from the patch-validation methods in Section 5.1.1: reproduction methods still solve only 49% of SWT-Bench Verified instances [96, 152]. In total, 49 of the 242 surveyed papers (20.2%) explicitly raise this challenge. Current LLM-based systems mainly rely on heuristic or prompt-based regression-test retrieval, which remains incomplete or redundant [110, 252]. TestLoc [28] shows that selecting relevant regression tests can improve reproduction test generation for new issues and strengthen patch validation, but reliably identifying such tests remains challenging. Future work should combine repository-aware reproduction-test generation with program-analysis-based regression-test selection [94], while addressing the unsafe omission of affected tests in current techniques [18, 94]. Advancing both problems calls for expertise in software testing, and mature solutions would immediately benefit the patch validation stage of nearly every system surveyed in this work.

7.4 Training Domain-Specific Models for Issue Resolution

This challenge follows from the learning strategies in Section 5.2 and the performance evidence in Section 6.2: backbone choice can outweigh scaffold differences, yet leading commercial models remain inaccessible in privacy-sensitive settings and open models still lag behind [9, 36]. In total, 65 of the 242 surveyed papers (26.9%) explicitly raise this challenge, the highest among the challenges we identify. HE-SNR [228] shows that token-level entropy predicts downstream SWE-bench performance more reliably than aggregate perplexity during mid-training. Future work should use such diagnostics to select data, identify ineffective checkpoints, and preserve reasoning capability during alignment. Existing fine-tuned models are typically trained on trajectories collected from a single scaffold [84, 252], so the learned policy silently binds to that scaffold's tool set and observation format and transfers poorly to others. We therefore advocate treating the scaffold interface as a randomization dimension

Just Accepted

during training [212], in analogy to domain randomization in robotics, exposing the model to varied tool sets and observation formats so that it acquires scaffold-agnostic resolution capability rather than scaffold-specific habits.

Current agentic RL predominantly uses outcome rewards [76, 137, 256, 269], which may reward successful patches produced through unreliable processes. Moreover, prevailing training pipelines keep only successful trajectories for supervision and discard failures [84, 159], although failed trajectories encode recurring dead-end patterns that can train critics to recognize and abandon unpromising explorations early. The phase-level signals discussed above, such as localization accuracy and reproduction test verdicts, are natural sources of process rewards. This avenue is best pursued by model providers and research groups with sufficient training resources, and the resulting open models chiefly serve organizations for which commercial APIs are not an option.

7.5 Domain-Specific Issue Resolution

This direction is motivated by both benchmark- and instance-level evidence presented in Sections 4 and 6.2. Specifically, different systems solve complementary subsets of tasks, while their performance degrades in settings beyond Python libraries [65, 149, 253, 272]. Moreover, 49 of the 242 surveyed papers (20.2%) explicitly call for further research in this direction. These indicate substantial room for domain-specific issue resolution along two axes. Along the bug-family axis, each family exhibits distinct symptoms and admits distinct verification oracles, motivating an upfront classifier that routes each issue to a pipeline specialized for that family. For example, cosmetic and interaction bugs demand visual observation and rendering-aware oracles, whereas calculation bugs are better served by input-space exploration and numerical assertions. Along the system-type axis, each type of system requires a different execution harness, calling for agents that integrate the domain-specific toolkit of each ecosystem as first-class tools. For example, resolving issues in web applications relies on browser automation, while mobile apps require device emulators [81], and GPU kernels in AI frameworks must be validated by end-to-end tests within the original framework [78]. Progress hinges on benchmarks stratified by bug family and system type, which make specialization gains measurable. Constructing such benchmarks falls to academic researchers, while domain toolkits are best contributed by practitioners within each ecosystem.

7.6 SE for Agentic Systems

This direction is motivated by the recurrent, diagnosable failure modes identified in Section 6.2 [30, 248]. In total, 26 of the 242 surveyed papers (10.7%) explicitly raise this direction. Software Engineering for Agentic Systems (SE4AS) should extend testing, debugging, versioning, and continuous integration to agent lifecycles, with tools that expose internal states and feed diagnostic evidence back into training [43]. We see particular promise in porting classical debugging techniques to agent trajectories. Counterfactual replay [44], which intervenes on a single step and re-executes the remainder of a trajectory, can isolate the minimal decisions responsible for a failure in the spirit of delta debugging. Statistics over large collections of successful and failed trajectories can rank suspicious action patterns, mirroring spectrum-based fault localization, and process invariants mined from normal runs, such as never editing a file before reading it, can be monitored at runtime to abort abnormal executions early [153]. Establishing this field is a joint undertaking, in which academia supplies methodologies and tooling while industrial teams operating agents in production ground them with real workloads and failure data.

> **RQ4 Summary:** Due to the limitations identified above, issue resolution may need to move beyond single-score evaluation and assess agentic systems in a fine-grained manner from perspectives such as security, efficiency, robustness, and code quality. It may also require more effective and multimodal knowledge representations for software repositories, more reliable reproduction test generation and regression test selection, process-oriented reinforcement learning for open-source models, domain-specific customization

Just Accepted

across bug families and system types, and dedicated software engineering methods for developing, debugging, and maintaining agentic systems.

8 Threats to Validity

Internal Validity. The main internal threat lies in the manual paper screening and classification process, where subjective judgment may exclude relevant studies or misclassify papers into study types and taxonomy categories. To mitigate this threat, two authors independently apply the inclusion and exclusion criteria using a conservative union rule that retains a paper if either author votes to include it. They also independently assign study types and apply the type-adapted QAC. For taxonomy construction, two authors jointly derive each initial taxonomy from 70% of the corresponding papers and independently label the remaining 30%. Cohen's κ is reported for the independent stages, and remaining disagreements are discussed with a third author. We further anchor the study type assignment to the inclusion criteria each paper satisfies, conduct searches over four databases, using full-text search where it is supported and title-based metadata search on DBLP, together with backward and forward snowballing to reduce the risk of missing relevant work, and release the paper lists of every filtering stage on the artifacts page for public scrutiny.

External Validity. Our conclusions may not generalize beyond the collection cutoff of May 31, 2026, as issue resolution evolves rapidly. We mitigate this threat by treating our findings as a snapshot of the field as of the cutoff date. Additionally, 50.8% of the collected papers are arXiv preprints that have not undergone peer review, which may affect taxonomies. We mitigate this threat by applying the same type-adapted quality assessment to all papers and by reconstructing the taxonomy from the peer-reviewed subset alone, which yields an unchanged top-level taxonomy.

9 Conclusion

In this survey, we present a comprehensive and systematic literature review of LLM-based agentic systems for issue resolution from three perspectives: benchmarks, techniques, and empirical studies, and distill the research opportunities they suggest. Across these perspectives, three higher-level conclusions emerge. First, effective design is conditional rather than universal. Benchmarks should be selected by evaluation purpose, pipeline-based scaffolds suit tight budgets and weaker backbones, agent-based scaffolds suit frontier backbones, and agentic reinforcement learning has become the dominant training paradigm. Second, the persistent limitation is the gap between measured and actual capability, since weak test suites, data contamination, and unreliable trajectories inflate reported resolution rates and bound the outcome rewards used for training. Moreover, even for issues counted as resolved, prevailing execution-based evaluation overlooks the software engineering quality of generated patches, such as their maintainability and conformance to project-specific design and architectural constraints. Third, the field is likely to evolve from maximizing a single resolution rate toward process-aware development and evaluation, including fine-grained multi-perspective assessment, process-level supervision, richer repository knowledge representations, domain-specific customization, and software engineering methods for agentic systems themselves. Together, these contributions position this survey as a timely and foundational reference that consolidates fragmented progress, clarifies emerging trends, and supports the development of next-generation research on automated issue resolution.

Just Accepted

Acknowledgments

This research is supported by the National Natural Science Foundation of China (No. 92582107), Zhejiang Provincial Natural Science Foundation of China (No. LZ25F020003) and the National Research Foundation under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings, conclusions, or recommendations

expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

References

[1] Rui Abreu, Peter Zoeteweij, and Arjan JC Van Gemund. 2007. On the Accuracy of Spectrum-Based Fault Localization. In *Testing: Academic and Industrial Conference Practice and Research Techniques – MUTATION*. 89–98.

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[3] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. DARS: Dynamic Action Re-Sampling to Enhance Coding Agent Performance by Adaptive Tree Traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 19808–19855.

[4] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. In *Forty-second International Conference on Machine Learning*.

[5] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. 2026. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.

[6] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. 2026. Reproduction Test Generation for Java SWE Issues. *arXiv preprint arXiv:2605.04320* (2026).

[7] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *arXiv preprint arXiv:2412.02883* (2024).

[8] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. *arXiv preprint arXiv:2410.06992* (2024).

[9] Anthropic. 2025. *Introducing Claude Sonnet 4.5.* https://www.anthropic.com/news/claude-sonnet-4-5 Accessed: 2025-10-09.

[10] Anthropic. 2026. Claude Fable 5. https://www.anthropic.com/claude/fable. Published July 1, 2026. Accessed July 2, 2026.

[11] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2025. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. In *The Thirteenth International Conference on Learning Representations*.

[12] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2026. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *Advances in Neural Information Processing Systems* 38 (2026).

[13] Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. 2026. SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale. In *Forty-third International Conference on Machine Learning*.

[14] Ibragim Badertdinov, Maria Trofimova, Yury Anapolskiy, Sergey Abramov, Karina Zainullina, Alexander Golubev, Sergey Polezhaev, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergey Skvortsov, Maxim Nekrashevich, Anton Shevtsov, and Boris Yangel. 2024. Scaling Data Collection for Training Software Engineering Agents. Nebius Blog.

[15] Nikita Benkovich and Vitalii Valkov. 2026. Agyn: A Multi-Agent System for Team-Based Autonomous Software Engineering. *arXiv preprint arXiv:2602.01465* (2026).

[16] Solomon Berhe, Marc Maynard, and Foutse Khomh. 2023. Maintenance Cost of Software Ecosystem Updates. *Procedia Computer Science* 220 (2023), 608–615.

[17] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering*. 2188–2200.

[18] Islem Bouzenia, Bajaj Piyush Krishan, and Michael Pradel. 2024. DyPyBench: A Benchmark of Executable Python Software. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 338–358.

[19] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 2846–2857.

[20] Xin Cao and Nan Yu. 2025. SIADAFIX: Issue Description Response for Adaptive Program Repair. *arXiv preprint arXiv:2510.16059* (2025).

[21] Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, et al. 2025. CWM: An Open-Weights LLM for Research on Code Generation with World Models. *arXiv preprint arXiv:2510.02387* (2025).

[22] Partha Chakraborty, Mahmoud Alfadel, and Meiyappan Nagappan. 2025. BLAZE: Cross-Language and Cross-Project Bug Localization via Dynamic Chunking and Hard Example Learning. *IEEE Transactions on Software Engineering* (2025).

[23] Jianming Chang, Xin Zhou, Lulu Lulu, David Lo, and Bixin Li. 2026. Bridging Bug Localization and Issue Fixing: A Hierarchical Localization Framework Leveraging Large Language Models. *IEEE Transactions on Software Engineering* (2026).

[24] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, et al. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *arXiv preprint arXiv:2406.01304* (2024).

Just Accepted

[25] Jialong Chen, Xander Xu, Hu Wei, Chuan Chen, and Bing Zhao. 2026. SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration. *arXiv preprint arXiv:2603.03823* (2026).
[26] Mouxiang Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, et al. 2026. SWE-Universe: Scale Real-World Verifiable Environments to Millions. *arXiv preprint arXiv:2602.02361* (2026).
[27] Silin Chen, Shaoxin Lin, Xiaodong Gu, Yuling Shi, Heng Lian, Longfei Yun, Dong Chen, Weiguo Sun, Lin Cao, and Qianxiang Wang. 2025. SWE-Exp: Experience-Driven Software Issue Resolution. *arXiv preprint arXiv:2507.23361* (2025).
[28] Yang Chen, Toufique Ahmed, Reyhaneh Jabbarvand, and Martin Hirzel. 2026. Can Old Tests Do New Tricks for Resolving SWE Issues? *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 141:1–141:22.
[29] Zhi Chen and Lingxiao Jiang. 2025. Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering*. 657–668.
[30] Zhi Chen, Wei Ma, and Lingxiao Jiang. 2026. Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents in Real-World GitHub Scenarios. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.
[31] Zimin Chen, Yue Pan, Siyu Lu, Jiayi Xu, Claire Le Goues, Martin Monperrus, and He Ye. 2025. Prometheus: Unified Knowledge Graphs for Issue Resolution in Multilingual Codebases. *arXiv preprint arXiv:2507.19942* (2025).
[32] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8697–8727.
[33] Zhilong Chen, Chengzong Zhao, Boyuan Chen, Dayi Lin, Yihao Chen, Arthur Leung, Gopi Krishnan Rajbahadur, Gustavo A Oliva, Haoxiang Zhang, Aaditya Bhatia, et al. 2025. RepoForge: Training a SOTA Fast-Thinking SWE Agent with an End-to-End Data Curation Pipeline Synergizing SFT and RL at Scale. *arXiv preprint arXiv:2508.01550* (2025).
[34] Runxiang Cheng, Michele Tufano, José Cambronero, Renyao Wei, Sherry Shi, Grant Uy, Pat Rondon, and Franjo Ivančić. 2026. Dynamic Cogeneration of Bug Reproduction Test in Agentic Program Repair. In *Proceedings of the 34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering Companion*.
[35] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821* (2025).
[36] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/
[37] Alejandro Cuadron, Pengfei Yu, Yang Liu, and Arpit Gupta. 2026. SABER: Small Actions, Big Errors — Safeguarding Mutating Steps in LLM Agents. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*.
[38] Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. 2025. Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards. *arXiv preprint arXiv:2506.11425* (2025).
[39] Hankun Dai, Maoquan Wang, Mengnan Qi, Yikai Zhang, Zijian Jin, Yongqiang Yao, Yufan Huang, Shengyu Fu, and Elsie Nallipogu. 2025. Lita: Light Agent Uncovers the Agentic Coding Capabilities of LLMs. *arXiv preprint arXiv:2509.25873* (2025).
[40] Le Deng, Zhonghao Jiang, Jialun Cao, Michael Pradel, and Zhongxin Liu. 2025. NoCode-bench: A Benchmark for Evaluating Natural Language-Driven Feature Addition. *arXiv preprint arXiv:2507.18130* (2025).
[41] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. 2026. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?. In *Forty-third International Conference on Machine Learning*.
[42] Yi-Xuan Deng, Xiaoqin Liu, Yi Zhang, Guo-Wei Yang, and Shuojin Yang. 2026. Your Code Agent Can Grow alongside You with Structured Memory. *arXiv preprint arXiv:2603.13258* (2026).
[43] Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. 2025. TRAIL: Trace Reasoning and Agentic Issue Localization. *arXiv preprint arXiv:2505.08638* (2025).
[44] Yifeng Ding and Lingming Zhang. 2026. SWE-Replay: Efficient Test-Time Scaling for Software Engineering Agents. *arXiv preprint arXiv:2601.22129* (2026).
[45] Yaxin Du, Yuzhu Cai, Yifan Zhou, Cheng Wang, Yu Qian, Xianghe Pang, Qian Liu, Yue Hu, and Siheng Chen. 2025. SWE-Dev: Evaluating and Training Autonomous Feature-Driven Software Development. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.
[46] Ryan Ehrlich, Bradley Brown, Jordan Juravsky, Ronald Clark, Christopher Ré, and Azalia Mirhoseini. 2025. CodeMonkeys: Scaling Test-Time Compute for Software Engineering. *arXiv preprint arXiv:2501.14723* (2025).
[47] Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M Zhang, Pinjia He, and Ahmed E Hassan. 2025. SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness under Resource Constraints. *arXiv preprint arXiv:2509.09853* (2025).
[48] Fabio James Fehr, Luca Franceschi, Giovanni Zappella, et al. 2025. CoRet: Improved Retriever for Code Editing. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 775–789.
[49] Zhiwei Fei, Yue Pan, Federica Sarro, Jidong Ge, Marc Liu, Vincent Ng, and He Ye. 2026. Echo: Graph-Enhanced Retrieval and Execution Feedback for Issue Reproduction Test Generation. *arXiv preprint arXiv:2603.07326* (2026).

Just Accepted

[50] Qiong Feng, Xiaotian Ma, Jiayi Sheng, Ziyuan Feng, Wei Song, and Peng Liang. 2025. Integrating Various Software Artifacts for Better LLM-Based Bug Localization and Program Repair. *ACM Transactions on Software Engineering and Methodology* (2025).
[51] Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, et al. 2026. daVinci-Env: Open SWE Environment Synthesis at Scale. *arXiv preprint arXiv:2603.13023* (2026).
[52] Ying Fu, Teng Wang, Shanshan Li, Jinyan Ding, Shulin Zhou, Zhouyang Jia, Wang Li, Yu Jiang, and Xiangke Liao. 2024. MissConf: LLM-Enhanced Reproduction of Configuration-Triggered Bugs. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 484–495.
[53] Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. 2025. When Agents Go Astray: Course-Correcting SWE Agents with PRMs. In *Workshop on Scaling Environments for Agents*.
[54] Jatin Ganhotra, Sami Serhan, Antonio Abu Nassar, Avraham Shinnar, Ziv Nevo, and Martin Hirzel. 2026. Resolving Java Code Repository Issues with iSWE Agent. *arXiv preprint arXiv:2603.11356* (2026).
[55] Pengfei Gao and Chao Peng. 2026. More with Less: An Empirical Study of Turn-Control Strategies for Efficient Coding Agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*. 1–12.
[56] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. 2025. Trae Agent: An LLM-Based Agent for Software Engineering with Test-Time Scaling. *arXiv preprint arXiv:2507.23370* (2025).
[57] Spandan Garg and Yufan Huang. 2026. Debug2Fix: Supercharging Coding Agents with Interactive Debugging Capabilities. *arXiv e-prints* (2026).
[58] Spandan Garg, Benjamin Steenhoek, and Yufan Huang. 2026. Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation. In *Proceedings of the IEEE/ACM International Conference on AI Engineering – Software Engineering for AI*.
[59] Anmol Gautam, Kishore Kumar, Adarsh Jha, Mukunda NS, and Ishaan Bhola. 2024. SuperCoder2.0: Technical Report on Exploring the Feasibility of LLMs as Autonomous Programmer. *arXiv preprint arXiv:2409.11190* (2024).
[60] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. 2026. Coding Agents Don't Know When to Act. *arXiv preprint arXiv:2605.07769* (2026).
[61] Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, et al. 2025. Training Long-Context, Multi-Turn Software Engineering Agents with Reinforcement Learning. *arXiv preprint arXiv:2508.03501* (2025).
[62] Chuanzhe Guo, Jingjing Wu, Sijun He, Yang Chen, Zhaoqi Kuang, Shilong Fan, Bingjin Chen, Siqi Bao, Jing Liu, Hua Wu, et al. 2026. MEnvAgent: Scalable Polyglot Environment Construction for Verifiable Software Engineering. In *Forty-third International Conference on Machine Learning*.
[63] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).
[64] Jiale Guo, Suizhi Huang, Mei Li, Dong Huang, Xingsheng Chen, Regina Zhang, Zhijiang Guo, Han Yu, Siu-Ming Yiu, Christian Jensen, et al. 2025. A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. *arXiv preprint arXiv:2510.09721* (2025).
[65] Lianghong Guo, Wei Tao, Runhan Jiang, Yanlin Wang, Jiachi Chen, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. 2025. OmniGIRL: A Multilingual and Multimodal Benchmark for GitHub Issue Resolution. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 24–46.
[66] Lianghong Guo, Yanlin Wang, Caihua Li, Wei Tao, Pengyu Yang, Jiachi Chen, Haoyu Song, Duyu Tang, and Zibin Zheng. 2026. SWE Data Construction, Automatically! *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 525–546.
[67] Xinshuai Guo, Jiayi Kuang, Linyue Pan, Yinghui Li, Yangning Li, Hai-Tao Zheng, Ying Shen, Di Yin, and Xing Sun. 2026. EvoConfig: Self-Evolving Multi-Agent Systems for Efficient Autonomous Environment Configuration. *arXiv preprint arXiv:2601.16489* (2026).
[68] Yifu Guo, Jiaye Lin, Huacan Wang, Yuzhen Han, Sen Hu, Ziyi Ni, Licheng Wang, and Mingguang Chen. 2026. SE-Agent: Self-Evolution Trajectory Optimization in Multi-Step Reasoning with LLM-Based Agents. *Advances in Neural Information Processing Systems* 38 (2026), 116314–116341.
[69] Yaoqi Guo, Ying Xiao, Jie M Zhang, Mark Harman, Yiling Lou, Yang Liu, and Zhenpeng Chen. 2026. EET: Experience-Driven Early Termination for Cost-Efficient Software Engineering Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. 33008–33024.
[70] Dhruv Gupta, Gayathri Ganesh Lakshmy, and Yiqing Xie. 2025. SACL: Understanding and Combating Textual Bias in Code Retrieval with Semantic-Augmented Reranking and Localization. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 25052–25065.
[71] Hao Han, Jin Xie, Xuehao Ma, Weiquan Zhu, Ziyao Zhang, ZhiLiang Long, Hongkai Chen, and Qingwen Ye. 2026. SWE-TRACE: Optimizing Long-Horizon SWE Agents through Rubric Process Reward Models and Heuristic Test-Time Scaling. *arXiv preprint arXiv:2604.14820* (2026).
[72] Kevin Han, Siddharth Maddikayala, Tim Knappe, Om Patel, Austen Liao, and Amir Barati Farimani. 2026. TDFlow: Agentic Workflows for Test Driven Development. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational*

Just Accepted

Linguistics (Volume 1: Long Papers). 1511–1527.
[73] Hiroaki Hayashi, Bo Pang, Wenting Zhao, Ye Liu, Akash Gokul, Srijan Bansal, Caiming Xiong, Semih Yavuz, and Yingbo Zhou. 2025. Self-Abstraction from Grounded Experience for Plan-Guided Policy Refinement. *arXiv preprint arXiv:2511.05931* (2025).
[74] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. *arXiv preprint arXiv:2603.01327* (2026).
[75] Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, ZhiJie Fan, Yiming Huang, Zejian Yuan, and Zejun MA. 2025. SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories?. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.
[76] Zhenyu He, Qingping Yang, Wei Sheng, Xiaojian Zhong, Kechi Zhang, Chenxin An, Wenlei Shi, Tianle Cai, Di He, Jiaze Chen, and Jingjing Xu. 2025. SWE-Swiss: A Multi-Task Fine-Tuning and RL Recipe for High-Performance Issue Resolution. https://www.notion.so/SWE-Swiss-A-Multi-Task-Fine-Tuning-and-RL-Recipe-for-High-Performance-Issue-Resolution-21e174dedd4880ea829ed4c861c44f88. Notion Blog.
[77] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
[78] Jinjun Huang, Zhongzhen Wen, Tongtong Xu, Meng Yan, Xin Xia, and Zhongxin Liu. 2026. RealisticTritonBench: A Benchmark for Triton-Kernel Generation in Real-World AI Frameworks. In *2026 41st IEEE/ACM International Conference on Automated Software Engineering*.
[79] Jiawei Huang, Qingping Yang, Renjie Zheng, and Jiaze Chen. 2026. Beyond Verifiable Rewards: Rubric-Based GRM for Reinforced Fine-Tuning SWE Agents. *arXiv preprint arXiv:2604.16335* (2026).
[80] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing Is Fixing: Cross-Modal Reasoning with Multimodal LLMs for Visual Software Issue Repair. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 1156–1168.
[81] Yuchao Huang, Junjie Wang, Zhe Liu, Yawen Wang, Song Wang, Chunyang Chen, Yuanzhe Hu, and Qing Wang. 2024. CrashTranslator: Automatically Reproducing Mobile Application Crashes Directly from Stack Trace. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
[82] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2025. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. In *The Thirteenth International Conference on Learning Representations*.
[83] Naman Jain, Manish Shetty, Tianjun Zhang, King Han, Koushik Sen, and Ion Stoica. 2024. R2E: Turning Any GitHub Repository into a Programming Agent Environment. In *Forty-first International Conference on Machine Learning*.
[84] Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.
[85] Haoxiang Jia, Earl T Barr, and Sergey Mechtaev. 2026. Compressing Code Context for LLM-Based Issue Resolution. *arXiv preprint arXiv:2603.28119* (2026).
[86] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–72.
[87] Mingjian Jiang, Yangjun Ruan, Luis Lastras, Pavan Kapanipathi, and Tatsunori Hashimoto. 2025. Putting It All into Context: Simplifying Agents with LCLMs. *arXiv preprint arXiv:2505.08120* (2025).
[88] Zhonghao Jiang, Le Deng, Jialun Cao, Michael Pradel, and Zhongxin Liu. 2026. Doc2Feat-Bench: Evaluating Documentation-Driven Feature Addition. In *2026 41st IEEE/ACM International Conference on Automated Software Engineering*.
[89] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3034–3045.
[90] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*.
[91] James A Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of Test Information to Assist Fault Localization. In *Proceedings of the 24th International Conference on Software Engineering*. 467–477.
[92] Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. 2025. SWE-Bench-CL: Continual Learning for Coding Agents. *arXiv preprint arXiv:2507.00014* (2025).
[93] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models Are Few-Shot Testers: Exploring LLM-Based General Bug Reproduction. In *45th IEEE/ACM International Conference on Software Engineering*. 2312–2323.
[94] Eero Kauhanen, Jukka K Nurminen, Tommi Mikkonen, and Matvei Pashkovskiy. 2021. Regression Test Selection Tool for Python in Continuous Integration Process. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering*. 618–621.
[95] Kabir Khandpur. 2025. SWE-bench Multilingual: Evaluating LLMs on Software Engineering across Multiple Programming Languages. Kabir Khandpur's Blog. https://kabirk.com/multilingual Accessed 2025-06-15.
[96] Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. 2026. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.

Just Accepted

[97] Joongwon Kim, Wannan Yang, Kelvin Niu, Hongming Zhang, Yun Zhu, Eryk Helenowski, Ruan Silva, Zhengxing Chen, Srinivasan Iyer, Manzil Zaheer, et al. 2026. Scaling Test-Time Compute for Agentic Coding. *arXiv preprint arXiv:2604.16529* (2026).
[98] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. 2026. CODESTRUCT: Code Agents over Structured Action Spaces. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13290–13306.
[99] Myeongsoo Kim, Dingmin Wang, Siwei Cui, Farima Farmahinifarahani, Shweta Garg, Baishakhi Ray, Terry Yue Zhuo, Rajdeep Mukherjee, and Varun Kumar. 2026. TRAJEVAL: Decomposing Code Agent Trajectories for Fine-Grained Diagnosis. *arXiv preprint arXiv:2603.24631* (2026).
[100] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic Literature Reviews in Software Engineering – A Systematic Literature Review. *Information and Software Technology* 51, 1 (2009), 7–15.
[101] Konstantinos Kitsios, Marco Castelluccio, and Alberto Bacchelli. 2025. Automated Generation of Issue-Reproducing Tests by Combining LLMs and Search-Based Testing. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 1982–1994.
[102] Patrick Tser Jern Kon, Archana Pradeep, Ang Chen, Alexander P Ellis, Warren Hunt, Zijian Wang, John Yang, and Samuel Thompson. 2026. SWE-Protégé: Learning to Selectively Collaborate with an Expert Unlocks Small Language Models as Software Engineering Agents. *arXiv preprint arXiv:2602.22124* (2026).
[103] Shiqi Kuang, Zhao Tian, Kaiwei Lin, Chaofan Tao, Shaowei Wang, Haoli Bai, Lifeng Shang, and Junjie Chen. 2026. REAgent: Requirement-Driven LLM Agents for Software Issue Resolution. *arXiv preprint arXiv:2604.06861* (2026).
[104] Hiroki Kuramoto, Dong Wang, Masanari Kondo, Yutaro Kashiwa, Yasutaka Kamei, and Naoyasu Ubayashi. 2024. Understanding the Characteristics and the Role of Visual Issue Reports. *Empirical Software Engineering* 29, 4 (2024), 89.
[105] Man Ho Lam, Chaozheng Wang, Hange Liu, Jingyu Xiao, Haau-sing Li, Jen-tse Huang, Terry Yue Zhuo, and Michael R Lyu. 2026. SWE-Chain: Benchmarking Coding Agents on Chained Release-Level Package Upgrades. *arXiv preprint arXiv:2605.14415* (2026).
[106] Bin Lei, Weitai Kang, Zijian Zhang, Winson Chen, Xi Xie, Shan Zuo, Mimi Xie, Ali Payani, Mingyi Hong, Yan Yan, et al. 2026. InfantAgent-Next: A Multimodal Generalist Agent for Automated Computer Interaction. *Advances in Neural Information Processing Systems* 38 (2026), 34494–34520.
[107] Bin Lei, Yuchen Li, Yiming Zeng, Tao Ren, Yi Luo, Tianyu Shi, Zitian Gao, Zeyu Hu, Weitai Kang, and Qiuwu Chen. 2024. Infant Agent: A Tool-Integrated, Logic-Driven Agent with Cost-Effective API Usage. *arXiv preprint arXiv:2411.01114* (2024).
[108] Chenglin Li, Yisen Xu, Zehao Wang, Shin Hwei Tan, et al. 2026. From Historical Patches to Repair Plans: Outcome-Conditioned Reasoning for Repository-Level Program Repair. In *Forty-third International Conference on Machine Learning*.
[109] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. 2026. SWE-Debate: Competitive Multi-Agent Debate for Software Issue Resolution. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.
[110] Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025. PatchPilot: A Stable and Cost-Efficient Agentic Patching Framework. *arXiv e-prints* (2025).
[111] Hanyu Li, Yichi Zhang, Speed Zhu, Hang Su, Jun Zhu, and Yinpeng Dong. 2026. RepoMirage: Probing Repository Context Reasoning in Code Agents with Perturbations. *arXiv preprint arXiv:2605.26177* (2026).
[112] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. *arXiv preprint arXiv:2602.05892* (2026).
[113] Kenan Li, Qirui Jin, Liao Zhu, Xiaosong Huang, Yijia Wu, Yikai Zhang, Xin Zhang, Zijian Jin, Yufan Huang, Elsie Nallipogu, et al. 2026. ORACLE-SWE: Quantifying the Contribution of Oracle Information Signals on SWE Agents. *arXiv preprint arXiv:2604.07789* (2026).
[114] KeFan Li, Mengfei Wang, Hengzhi Zhang, Zhichao Li, Yuan Yuan, Mu Li, Xiang Gao, Hailong Sun, Chunming Hu, and Weifeng Lv. 2025. InfCode: Adversarial Iterative Refinement of Tests and Patches for Reliable Software Issue Resolution. *arXiv preprint arXiv:2511.16004* (2025).
[115] Kefan Li, Yuan Yuan, Mengfei Wang, Shihao Zheng, Wei Wang, Ping Yang, Mu Li, and Weifeng Lv. 2026. Beyond Fixed Tests: Repository-Level Issue Resolution as Coevolution of Code and Behavioral Constraints. *arXiv preprint arXiv:2604.04580* (2026).
[116] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. 2025. FEA-Bench: A Benchmark for Evaluating Repository-Level Code Generation for Feature Implementation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17160–17176.
[117] Yuanhao Li, Hongbo Wang, Xiaotang Shang, Xunzhu Tang, Yiming Cao, and Xuhong Chen. 2026. BoostAPR: Boosting Automated Program Repair via Execution-Grounded Reinforcement Learning with Dual Reward Models. In *Forty-third International Conference on Machine Learning*.
[118] Shuquan Lian, Juncheng Liu, Yazhe Chen, Yuhong Chen, and Hui Li. 2026. SWE-AGILE: A Software Agent Framework for Efficiently Managing Dynamic Reasoning Context. In *Findings of the Association for Computational Linguistics: ACL 2026*. 17536–17550.
[119] Jiarong Liang, Zhiheng Lyu, Zijie Liu, Xiangchao Chen, Ping Nie, Kai Zou, and Wenhu Chen. 2026. SWE-Next: Scalable Real-World Software Engineering Tasks for Agents. *arXiv preprint arXiv:2603.20691* (2026).

Just Accepted

[120] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. 2026. The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. 395–405.

[121] Bennet P Lientz, E. Burton Swanson, and Gail E Tompkins. 1978. Characteristics of Application Software Maintenance. *Commun. ACM* 21, 6 (1978), 466–471.

[122] Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. LLMs as Continuous Learners: Improving the Reproduction of Defective Code in Software Issues. *arXiv preprint arXiv:2411.13941* (2024).

[123] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Transactions on Software Engineering and Methodology* (2026).

[124] Mingwei Liu, Zhenxi Chen, Zheng Pei, Zihao Wang, Yanlin Wang, and Zibin Zheng. 2026. Architecture-Aware Multi-Design Generation for Repository-Level Feature Addition. *arXiv preprint arXiv:2603.01814* (2026).

[125] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[126] Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhaneh Jabbarvand. 2026. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages* 10, OOPSLA1 (2026), 1961–1988.

[127] Shuyang Liu, Saman Dehghan, Jatin Ganhotra, Martin Hirzel, and Reyhaneh Jabbarvand. 2026. From Plan to Action: How Well Do Agents Follow the Plan? *arXiv e-prints* (2026).

[128] Simiao Liu, Fang Liu, Liehao Li, Xin Tan, Yinghao Zhu, Xiaoli Lian, and Li Zhang. 2025. An Empirical Study on Failures in Automated Issue Solving. *arXiv preprint arXiv:2509.13941* (2025).

[129] Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. 2026. Context as a Tool: Context Management for Long-Horizon SWE-agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. 20604–20617.

[130] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2026. GraphLocator: Graph-Guided Causal Reasoning for Issue Localization. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1128–1149.

[131] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Qizhe Shieh, and Wenmeng Zhou. 2025. CodeXGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 142–160.

[132] Yizhou Liu, Pengfei Gao, Xinchen Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. MarsCode Agent: AI-Native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899* (2024).

[133] Zhongxin Liu, Zhonghao Jiang, Zhifan Ye, Haoye Wang, Jiakun Liu, and Xiaoxue Ren. 2026. Effective and Efficient Context Retrieval via Partial Dependency Graph for Repository-Level Code Generation. *arXiv preprint arXiv:2608.01927* (2026).

[134] LiveSWEBench Team. 2025. LiveSWEBench: A Contamination-Free Benchmark for AI Software Engineers. Official Website. https://liveswebench.ai/ Accessed 2025-06-14.

[135] Nikolai Ludwig, Wasi Uddin Ahmad, Somshubra Majumdar, and Boris Ginsburg. 2026. From SWE-ZERO to SWE-HERO: Execution-Free to Execution-Based Fine-Tuning for Software Engineering Agents. *arXiv preprint arXiv:2604.01496* (2026).

[136] Jane Luo, Chengyu Yin, Xin Zhang, Qingtao Li, Steven Liu, Yiming Huang, Jie Wu, Hao Liu, Yangyu Huang, Yu Kang, et al. 2026. Closing the Loop: Universal Repository Representation with RPG-Encoder. In *Forty-third International Conference on Machine Learning*.

[137] Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Shang Zhu Tarun Venkat, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. 2025. DeepSWE: Training a State-of-the-Art Coding Agent from Scratch by Scaling RL. https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902c1468193aabbe9a8c59bbe33. Notion Blog.

[138] Jeffrey Jian Ma, Milad Hashemi, Amir Yazdanbakhsh, Kevin Swersky, Ofir Press, Enhui Li, Vijay Janapa Reddi, and Parthasarathy Ranganathan. 2026. SWE-fficiency: Can Language Models Optimize Real-World Repositories on Real Workloads?. In *Forty-third International Conference on Machine Learning*.

[139] Ruize Ma, Yilei Jiang, Shilin Zhang, Zheng Ma, Yi Feng, Vincent Ng, Zhi Wang, Xiangyu Yue, Chuanyi Li, and Lewei Lu. 2026. FailureMem: A Failure-Aware Multimodal Framework for Autonomous Software Repair. *arXiv preprint arXiv:2603.17826* (2026).

[140] Yingwei Ma, Rongyu Cao, Yongchang Cao, Yue Zhang, Jue Chen, Yibo Liu, Yuchen Liu, Binhua Li, Fei Huang, and Yongbin Li. 2025. SWE-GPT: A Process-Centric Language Model for Automated Software Improvement. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2362–2383.

[141] Yingwei Ma, Yongbin Li, Yihong Dong, Xue Jiang, Yanhao Li, Yue Liu, Rongyu Cao, Jue Chen, Fei Huang, and Binhua Li. 2025. Thinking Longer, Not Larger: Enhancing Software Engineering Agents via Scaling Test-Time Compute. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 3730–3741.

[142] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 238–249.

Just Accepted

[143] Zexiong Ma, Shengnan An, Zeqi Lin, Yanzhen Zou, and Bing Xie. 2024. Repository Structure-Aware Training Makes SLMs Better Issue Resolver. *arXiv preprint arXiv:2412.19031* (2024).
[144] Zexiong Ma, Chao Peng, Pengfei Gao, Xiangxin Meng, Yanzhen Zou, and Bing Xie. 2025. SoRFT: Issue Resolving with Subtask-Oriented Reinforced Fine-Tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11427–11441.
[145] Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025. Tool-Integrated Reinforcement Learning for Repo Deep Search. *arXiv preprint arXiv:2508.03012* (2025).
[146] Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. 2025. Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories. *arXiv preprint arXiv:2511.00197* (2025).
[147] Md Afif Al Mamun and Gias Uddin. 2026. BLAgent: Agentic RAG for File-Level Bug Localization. *ACM Transactions on Software Engineering and Methodology* (2026).
[148] Matias Martinez and Xavier Franch. 2026. What's in a Benchmark? The Case of SWE-Bench in Automated Program Repair. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*. 1–12.
[149] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. 2024. An Empirical Study on LLM-Based Agents for Automated Bug Fixing. *arXiv preprint arXiv:2411.10213* (2024).
[150] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. 2026. LLM-Based Agents for Automated Bug Fixing: How Far Are We?. In *2026 IEEE/ACM 48th International Conference on Software Engineering*.
[151] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2026. ExpeRepair: Dual-Memory Enhanced LLM-Based Repository-Level Program Repair. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), FSE174:1–FSE174:23.
[152] Niels Mündler, Mark Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. *Advances in Neural Information Processing Systems* 37 (2024), 81857–81887.
[153] Rahul Nanda, Chandra Maddila, Smriti Jha, Euna Mehnaz Khan, Matteo Paltenghi, and Satish Chandra. 2026. Wink: Recovering from Misbehaviors in Coding Agents. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*. 208–217.
[154] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2026. Issue2Test: Generating Reproducing Test Cases from Issue Reports. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.
[155] Feifei Niu, Chuanyi Li, Kui Liu, Xin Xia, and David Lo. 2025. When Deep Learning Meets Information Retrieval-Based Bug Localization: A Survey. *Comput. Surveys* 57, 11 (2025), 1–41.
[156] Gustavo A. Oliva, Gopi Krishnan Rajbahadur, Aaditya Bhatia, Haoxiang Zhang, Yihao Chen, Zhilong Chen, Arthur Leung, Dayi Lin, Boyuan Chen, and Ahmed E. Hassan. 2025. SPICE: An Automated SWE-Bench Labeling Pipeline for Issue Clarity, Test Coverage, and Effort Estimation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 2325–2337.
[157] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *The Thirteenth International Conference on Learning Representations*.
[158] Anvith Pabba, Alex Mathai, Anindya Chakraborty, and Baishakhi Ray. 2025. SemAgent: A Semantics Aware Program Repair Agent. *arXiv preprint arXiv:2506.16650* (2025).
[159] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. Training Software Engineering Agents and Verifiers with SWE-Gym. In *Forty-second International Conference on Machine Learning*.
[160] Zhongqiang Pan, Chuanyi Li, Wenkang Zhong, Yi Feng, Bin Luo, and Vincent Ng. 2026. RepoRepair: Leveraging Code Documentation for Repository-Level Automated Program Repair. *arXiv preprint arXiv:2603.01048* (2026).
[161] Adamenko Pavel, Ivanov Mikhail, Aidar Valeev, Rodion Levichev, Pavel Zadorozhny, Ivan Lopatin, Dmitrii Babaev, Alena Fenogenova, and Valentin Malykh. 2025. SWE-MERA: A Dynamic Benchmark for Agenticly Evaluating Large Language Models on Software Engineering Tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 440–452.
[162] Jinjun Peng, Magnus Saebo, Tianjun Zhong, Yi-Jie Cheng, Junfeng Yang, Baishakhi Ray, Simin Chen, and Yangruibo Ding. 2026. SWE-Spot: Building Small Repo-Experts with Repository-Centric Learning. *arXiv preprint arXiv:2601.21649* (2026).
[163] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for Conducting Systematic Mapping Studies in Software Engineering: An Update. *Information and Software Technology* 64 (2015), 1–18.
[164] Minh VT Pham, Huy N Phan, Hoang N Phan, Cuong Le Chi, Tien N Nguyen, and Nghi DQ Bui. 2025. SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs. *arXiv preprint arXiv:2504.14757* (2025).
[165] Huy Nhat Phan, Tien N Nguyen, Phong X Nguyen, and Nghi DQ Bui. 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. *arXiv preprint arXiv:2409.16299* (2024).
[166] Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. 2025. Does SWE-Bench-Verified Test Agent Ability or Model Memory? *arXiv preprint arXiv:2512.10218* (2025).
[167] Mohit Raghavendra, Anisha Gunjal, Bing Liu, and Yunzhong He. 2026. Agentic Rubrics as Contextual Verifiers for SWE Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15265–15290.

Just Accepted

[168] Hanif Ur Rahman, Alberto Rodrigues da Silva, Asaad Alzayed, and Mushtaq Raza. 2024. A Systematic Literature Review on Software Maintenance Offshoring Decisions. *Information and Software Technology* 172 (2024), 107475.
[169] Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, et al. 2025. SWE-PolyBench: A Multi-Language Benchmark for Repository Level Evaluation of Coding Agents. *arXiv preprint arXiv:2504.08703* (2025).
[170] Abhinav Rastogi, Adam Yang, Albert Q Jiang, Alexander H Liu, Alexandre Sablayrolles, Amélie Héliou, Amélie Martin, Anmol Agarwal, Andy Ehrenberg, Andy Lo, et al. 2025. Devstral: Fine-Tuning Language Models for Coding Agent Applications. *arXiv preprint arXiv:2509.25193* (2025).
[171] Revanth Gangi Reddy, Ye Liu, Wenting Zhao, JaeHyeok Doo, Tarun Suresh, Daniel Lee, Caiming Xiong, Yingbo Zhou, Semih Yavuz, and Shafiq Joty. 2025. SweRank+: Multilingual, Multi-Turn Code Ranking for Software Issue Localization. *arXiv preprint arXiv:2512.20482* (2025).
[172] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan-Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2026. SweRank: Software Issue Localization with Code Ranking. In *The Fourteenth International Conference on Learning Representations*.
[173] Stephen Robertson, Hugo Zaragoza, et al. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
[174] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. 2025. A Self-Improving Coding Agent. In *Scaling Self-Improving Foundation Models without Human Supervision*.
[175] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2025. SpecRover: Code Intent Extraction via LLMs. In *2025 IEEE/ACM 47th International Conference on Software Engineering*. 963–974.
[176] Priyam Sahoo, Gaurav Mittal, Xiaomin Li, Shengjie Ma, Benjamin Steenhoek, Pingping Lin, and Yu Hu. 2026. AgentLens: Revealing the Lucky Pass Problem in SWE-agent Evaluation. *arXiv preprint arXiv:2605.12925* (2026).
[177] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. Are AI-Generated Fixes Secure? Analyzing LLM and Agent Patches on SWE-bench. *arXiv preprint arXiv:2507.02976* (2025).
[178] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. 2026. AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges. *Information Fusion* 126 (2026), 103599.
[179] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
[180] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on Software Engineering* 25, 4 (1999), 557–572.
[181] Shahd Seddik and Fatemeh Fard. 2026. ARISE: A Repository-Level Graph Representation and Toolset for Agentic Fault Localization and Program Repair. *arXiv preprint arXiv:2605.03117* (2026).
[182] Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. 2026. RGFL: Reasoning Guided Fault Localization for Automated Program Repair Using Large Language Models. *arXiv preprint arXiv:2601.18044* (2026).
[183] KN Shastry, Ganesh Senrayan, Shrey Satapara, Pranoy Panda, and Chaitanya Devaguptapu. 2026. Beyond Isolated Tasks: A Framework for Evaluating Coding Agents on Sequential Software Evolution. *arXiv preprint arXiv:2604.03035* (2026).
[184] Ethan Shen, Danny Tormoen, Saurabh Shah, Ali Farhadi, and Tim Dettmers. 2026. SERA: Soft-Verified Efficient Repository Agents. In *Forty-third International Conference on Machine Learning*.
[185] Manish Shetty, Naman Jain, Jinjian Liu, Vijay Kethanaboyina, Koushik Sen, and Ion Stoica. 2026. GSO: Challenging Software Optimization Tasks for Evaluating SWE-agents. *Advances in Neural Information Processing Systems* 38 (2026).
[186] Disha Shrivastava, Denis Kocetkov, Harm De Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. RepoFusion: Training Code Models to Understand Your Repository. *arXiv preprint arXiv:2306.10998* (2023).
[187] KaShun SHUM, Binyuan Hui, Jiawei Chen, Lei Zhang, X. W., Jiaxi Yang, Yuzhen Huang, Junyang Lin, and Junxian He. 2026. SWE-RM: Execution-Free Feedback for Software Engineering Agents. In *The Fourteenth International Conference on Learning Representations*.
[188] Atefeh Sohrabizadeh, Jialin Song, Mingjie Liu, Rajarshi Roy, Chankyu Lee, Jonathan Raiman, and Bryan Catanzaro. 2025. Nemotron-CORTEXA: Enhancing LLM Agents for Software Engineering Tasks via Improved Localization and Solution Diversity. In *Forty-second International Conference on Machine Learning*.
[189] Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, et al. 2026. SWE-Master: Unleashing the Potential of Software Engineering Agents via Post-Training. *arXiv preprint arXiv:2602.03411* (2026).
[190] Aditya Bharat Soni, Rajat Ghosh, Vaishnavi Bhargava, Valerie Chen, and Debojyoti Dutta. 2026. SWE-Tester: Training Open-Source LLMs for Issue Reproduction in Real-World Repositories. *arXiv preprint arXiv:2601.13713* (2026).
[191] Aditya Bharat Soni, Boxuan Li, Xingyao Wang, Valerie Chen, and Graham Neubig. 2025. Coding Agents with Multimodal Browsing Are Generalist Problem Solvers. In *ICML 2025 Workshop on Computer Use Agents*.

Just Accepted

[192] Atharv Sonwane, Isadora White, Hyunji Lee, Matheus Pereira, Lucas Caccia, Minseon Kim, Zhengyan Shi, Chinmay Singh, Alessandro Sordoni, Marc-Alexandre Côté, et al. 2025. BugPilot: Complex Bug Generation for Efficient Learning of SWE Skills. *arXiv preprint arXiv:2510.19898* (2025).

[193] Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Arik. 2025. Learn-by-Interact: A Data-Centric Framework for Self-Adaptive Agents in Realistic Environments. In *The Thirteenth International Conference on Learning Representations*.

[194] Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, et al. 2026. SWE-World: Building Software Engineering Agents in Docker-Free Environments. *arXiv preprint arXiv:2602.03419* (2026).

[195] Yuxuan Sun, Yuze Zhao, Yufeng Wang, Yao Du, Zhiyuan Ma, Jinbo Wang, Mengdi Zhang, Kai Zhang, and Zhenya Huang. 2026. SWE-Mutation: Can LLMs Generate Reliable Test Suites in Software Engineering?. In *Findings of the Association for Computational Linguistics: ACL 2026*. 39651–39674.

[196] Tarun Suresh, Revanth Gangi Reddy, Yifei Xu, Zach Nussbaum, Andriy Mulyar, Brandon Duderstadt, and Heng Ji. 2025. CoRNStack: High-Quality Contrastive Data for Better Code Retrieval and Reranking. In *The Thirteenth International Conference on Learning Representations*.

[197] Manan Suri, Xiangci Li, Mehdi Shojaie, Songyang Han, Chao-Chun Hsu, Shweta Garg, Aniket Anand Deshmukh, and Varun Kumar. 2026. CodeScout: Contextual Problem Statement Enhancement for Software Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. 40902–40931.

[198] Lintang Sutawika, Aditya Bharat Soni, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, Graham Neubig, et al. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. *arXiv preprint arXiv:2603.17829* (2026).

[199] Xunzhu Tang, Jiechao Gao, Jin Xu, Tiezhu Sun, Yewei Song, Saad Ezzini, Wendkûuni C Ouédraogo, Jacques Klein, and Tegawendé F Bissyandé. 2025. SynFix: Dependency-Aware Program Repair via RelationGraph Analysis. In *Findings of the Association for Computational Linguistics: ACL 2025*. 4878–4894.

[200] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. 2025. Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving. *arXiv preprint arXiv:2507.06229* (2025).

[201] Xiaoxuan Tang, Jincheng Wang, Liwei Luo, Jingxuan Xu, Sheng Zhou, Dajun Chen, Wei Jiang, and Yong Li. 2026. SVRepair: Structured Visual Reasoning for Automated Program Repair. *arXiv preprint arXiv:2602.06090* (2026).

[202] Yuheng Tang, Hongwei Li, Kaijie Zhu, Michael Yang, Yangruibo Ding, and Wenbo Guo. 2026. Co-PatcheR: Collaborative Software Patching with Component-Specific Small Reasoning Models. *Advances in Neural Information Processing Systems* 38 (2026), 42041–42069.

[203] Chaofan Tao, Jierun Chen, Yuxin Jiang, Kaiqi Kou, Shaowei Wang, Ruoyu Wang, Xiaohui Li, Sidi Yang, Yiming Du, Jianbo Dai, et al. 2026. SWE-Lego: Pushing the Limits of Supervised Fine-Tuning for Software Issue Resolving. *arXiv preprint arXiv:2601.01426* (2026).

[204] Hongyuan Tao, Ying Zhang, Zhenhao Tang, Hongen Peng, Xukun Zhu, Bingchang Liu, Yingguang Yang, Ziyin Zhang, Zhaogui Xu, Haipeng Zhang, et al. 2026. Code Graph Model (CGM): A Graph-Integrated Large Language Model for Repository-Level Software Engineering Tasks. *Advances in Neural Information Processing Systems* 38 (2026), 15869–15909.

[205] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. *Advances in Neural Information Processing Systems* 37 (2024), 51963–51993.

[206] Yicheng Tao, Yao Qin, and Yepang Liu. 2025. Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches. *arXiv preprint arXiv:2510.04905* (2025).

[207] Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. 2025. Meta-RAG on Large Codebases Using Code Summarization. *arXiv preprint arXiv:2508.02611* (2025).

[208] CodeArts Model Team, Yang Ye, Jingyuan Tan, Tianyue Jiang, Ruizhe Ye, Qiankun He, Jiarui Yang, Jian Dong, Sicong Liang, Chongjian Yue, et al. 2026. Yet Even Less Is Even Better for Agentic, Reasoning, and Coding LLMs. *arXiv preprint arXiv:2604.00824* (2026).

[209] Minh VT Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi DQ Bui. 2025. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios. *arXiv preprint arXiv:2512.18470* (2025).

[210] Muxin Tian, Zhe Wang, Blair Yang, Zhenwei Tang, Kunlun Zhu, Honghua Dong, Hanchen Li, Xinni Xie, Guangjing Wang, and Jiaxuan You. 2026. SWE-Bench Mobile: Can Large Language Model Agents Develop Industry-Level Mobile Applications?. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. 8077–8087.

[211] Zhao Tian, Pengfei Gao, Junjie Chen, and Chao Peng. 2026. Agent-Based Ensemble Reasoning for Repository-Level Issue Resolution. In *2026 IEEE/ACM 48th International Conference on Software Engineering*. 13.

[212] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 23–30.

[213] Arihant Tripathy, Ch Pavan Harshit, and Karthik Vaidhyanathan. 2026. SWEnergy: An Empirical Study on Energy Efficiency in Agentic Issue Resolution Frameworks with SLMs. In *Proceedings of the 2026 International Workshop on Agentic Engineering*. 104–111.

Just Accepted

[214] Konstantinos Vergopoulos, Mark Niklas Mueller, and Martin Vechev. 2025. Automated Benchmark Generation for Repository-Level Coding Tasks. In *Forty-second International Conference on Machine Learning*.
[215] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. 2026. Ambig-SWE: Interactive Agents to Overcome Underspecificity in Software Engineering. In *The Fourteenth International Conference on Learning Representations*.
[216] Nalin Wadhwa, Atharv Sonwane, Daman Arora, Abhav Mehrotra, Saiteja Utpala, Ramakrishna B Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. MASAI: Modular Architecture for Software-Engineering AI Agents. In *NeurIPS 2024 Workshop on Open-World Agents*.
[217] Boshi Wang, Weijian Xu, Yunsheng Li, Xuemei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. 2026. Improving Code Localization with Repository Memory. In *The Fourteenth International Conference on Learning Representations*.
[218] Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. 2025. SWE-Dev: Building Software Engineering Agents with Training and Inference Scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*. 3742–3761.
[219] Junyi Wang, Jialun Cao, and Zhongxin Liu. 2026. iCoRe: An Iterative Correlation-Aware Retriever for Bug Reproduction Test Generation. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), FSE186:1–FSE186:26.
[220] Junhao Wang, Daoguang Zan, Shulin Xin, Siyao Liu, Yurong Wu, and Kai Shen. 2025. SWE-Mirror: Scaling Issue-Resolving Datasets by Mirroring Issues across Repositories. *arXiv preprint arXiv:2509.08724* (2025).
[221] Lilin Wang, Lucas Ramalho, Alan Celestino, Phuc Anthony Pham, Yu Liu, Umang Kumar Sinha, Andres Portillo, Onassis Osunwa, and Gabriel Maduekwe. 2025. SWE-Bench++: A Framework for the Scalable Generation of Software Engineering Benchmarks from Open-Source Repositories. *arXiv preprint arXiv:2512.17419* (2025).
[222] Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, Heng Lian, Kunyi Wang, Xiaoming Yu, Jianghao Yin, Sen Hu, et al. 2026. MemGovern: Enhancing Code Agents through Learning from Governed Human Experiences. *arXiv preprint arXiv:2601.06789* (2026).
[223] Shengjie Wang, Guanghe Li, Zonghan Yang, and Yang Gao. 2026. Hindsight Hint Distillation: Scaffolded Reasoning for SWE Agents from CoT-Free Answers. *arXiv preprint arXiv:2605.11556* (2026).
[224] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. 2026. Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine. In *The Fourteenth International Conference on Learning Representations*.
[225] Xingyao Wang, Valerie Chen, Heng Ji, and Graham Neubig. 2026. A Rubric-Supervised Critic from Sparse Real-World Outcomes. *arXiv preprint arXiv:2603.03800* (2026).
[226] Xinchen Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. 2025. AEGIS: An Agent-Based Framework for Bug Reproduction from Issue Descriptions. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 331–342.
[227] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*.
[228] Yueyang Wang, Jiawei Fu, Baolong Bi, Xili Wang, and Xiaoqing Liu. 2026. HE-SNR: Uncovering Latent Logic via Entropy for Guiding Mid-Training on SWE-BENCH. In *Forty-third International Conference on Machine Learning*.
[229] Yibo Wang, Zhihao Peng, Ying Wang, Zhao Wei, Hai Yu, and Zhiliang Zhu. 2025. MCTS-Refined CoT: High-Quality Fine-Tuning Data for LLM-Based Repository Issue Resolution. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering*. 1844–1855.
[230] You Wang, Michael Pradel, and Zhongxin Liu. 2026. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.
[231] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. *arXiv preprint arXiv:2601.16746* (2026).
[232] Zhaodong Wang, Zhenting Qi, Sherman Wong, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Yining Yang, Ben Maurer, Wenlin Chen, et al. 2025. Confucius Code Agent: An Open-Sourced AI Software Engineer at Industrial Scale. *arXiv preprint arXiv:2512.10398* (2025).
[233] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida Wang. 2026. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *Advances in Neural Information Processing Systems* 38 (2026), 78500–78525.
[234] Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. 2026. Toward Training Superintelligent Software Agents through Self-Play SWE-RL. In *Forty-third International Conference on Machine Learning*.
[235] Xin-Cheng Wen, Binbin Chen, Haoxuan Lan, Hang Yu, Peng Di, and Cuiyun Gao. 2026. SWE-Fuse: Empowering Software Agents via Issue-Free Trajectory Learning and Entropy-Aware RLVR Training. *arXiv preprint arXiv:2603.07927* (2026).
[236] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
[237] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? *arXiv preprint arXiv:2511.13646* (2025).

Just Accepted

[238] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each Using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.

[239] Jiahong Xiang, Wenxiao He, Xihua Wang, Hongliang Tian, and Yuqun Zhang. 2026. Evaluating and Improving Automated Repository-Level Rust Issue Resolution with LLM-Based Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering*. 13.

[240] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), FSE056:1–FSE056:22.

[241] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139.

[242] Yiqing Xie, Emmy Liu, Gaokai Zhang, Nachiket Kotalwar, Shubham Gandhi, Sathwik Acharya, Xingyao Wang, Carolyn Rose, Graham Neubig, and Daniel Fried. 2026. Hybrid-Gym: Training Coding Agents to Generalize across Tasks. In *Forty-third International Conference on Machine Learning*.

[243] Bojian Xiong, Yikun Lei, Xikai Liu, Shaowei Zhang, Pengyun Zhu, Yan Liu, Yongqi Leng, Ling Shi, Meizhi Zhong, Yurong Zhang, et al. 2025. Think-Search-Patch: A Retrieval-Augmented Reasoning Framework for Repository-Level Code Repair. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 1555–1566.

[244] Ke Xu, Siyang Xiao, Ming Liang, Yichen Yu, Zhixiang Wang, Jingxuan Xu, Dajun Chen, Wei Jiang, and Yong Li. 2026. FuseSearch: Learning Adaptive Parallel Execution for Efficient Code Localization. In *Findings of the Association for Computational Linguistics: ACL 2026*. 2954–2967.

[245] Wendong XU, Jing Xiong, Chenyang Zhao, Qiujiang Chen, Haoran Wang, Hui Shen, Zhongwei Wan, Jianbo Dai, Taiqiang Wu, He Xiao, Chaofan Tao, Zhuoqing Mao, Ying Sheng, Zhijiang Guo, Hongxia Yang, Bei Yu, Lingpeng Kong, Quanquan Gu, and Ngai Wong. 2026. SWINGARENA: Adversarial Programming Arena for Long-Context GitHub Issue Solving. In *The Fourteenth International Conference on Learning Representations*.

[246] Xiufeng Xu, Xiufeng Wu, Zejun Zhang, and Yi Li. 2026. Neurosymbolic Repo-Level Code Localization. *arXiv preprint arXiv:2604.16021* (2026).

[247] Yisen Xu, Jinqiu Yang, et al. 2026. SWE-Refactor: A Repository-Level Benchmark for Real-World LLM-Based Code Refactoring. *arXiv preprint arXiv:2602.03712* (2026).

[248] Haoran Xue, Gias Uddin, and Song Wang. 2025. PAGENT: Learning to Patch Software Engineering Agents. *arXiv preprint arXiv:2506.17772* (2025).

[249] Aashish Yadavally, Hoan Nguyen, Laurent Callot, and Gauthier Guinet. 2025. Large Language Model Critics for Execution-Free Evaluation of Code Changes. *arXiv preprint arXiv:2501.16655* (2025).

[250] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F Bissyandé, Yang Liu, and Haoye Tian. 2025. A Survey of LLM-Based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. *arXiv preprint arXiv:2506.23749* (2025).

[251] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. *arXiv preprint arXiv:2503.21710* (2025).

[252] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[253] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. 2025. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?. In *The Thirteenth International Conference on Learning Representations*.

[254] John Yang, Kilian Lieret, Carlos Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2026. SWE-smith: Scaling Data for Software Engineering Agents. *Advances in Neural Information Processing Systems* 38 (2026).

[255] Xu Yang, Jiayuan Zhou, Michael Pacheco, Wenhan Zhu, Pengfei He, Shaowei Wang, Kui Liu, and Ruiqi Pan. 2025. Lingxi: Repository-Level Issue Resolution Framework Enhanced by Procedural Knowledge Guided Scaling. *arXiv preprint arXiv:2510.11838* (2025).

[256] Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, YINGWEI MA, Yanhao Li, Yue Liu, Zhenxing Hu, kaitai zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, Zhilin Yang, and Tianyu Liu. 2026. Kimi-Dev: Agentless Training as Skill Prior for SWE-agents. In *The Fourteenth International Conference on Learning Representations*.

[257] Boxi Yu, Yang Cao, Yuzhong Zhang, Liting Lin, Junjielong Xu, Zhiqing Zhong, Qinghua Xu, Guancheng Wang, Jialun Cao, Shing-Chi Cheung, et al. 2026. SWE-ABS: Adversarial Benchmark Strengthening Exposes Inflated Success Rates on Test-Based Benchmark. In *Forty-third International Conference on Machine Learning*.

[258] Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. 2025. UTBoost: Rigorous Evaluation of Coding Agents on SWE-bench. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3762–3774.

[259] Jiahao Yu, Zelei Cheng, Xian Wu, and Xinyu Xing. 2025. Building Coding Agents via Entropy-Enhanced Multi-Turn Preference Optimization. *arXiv preprint arXiv:2509.12434* (2025).

Just Accepted

[260] Kai Yu, Zhenhao Zhou, Junhao Zeng, Ying Wang, Xueying Du, Zhiqiang Yuan, Junwei Liu, Ziyu Zhou, Yujia Wang, Chong Wang, et al. 2026. Does Pass Rate Tell the Whole Story? Evaluating Design Constraint Compliance in LLM-Based Issue Resolution. *arXiv preprint arXiv:2604.05955* (2026).

[261] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. 2025. OrcaLoca: An LLM Agent Framework for Software Issue Localization. In *Forty-second International Conference on Machine Learning*.

[262] Danlong Yuan, Wei Wu, Zhengren Wang, Xueliang Zhao, Huishuai Zhang, and Dongyan Zhao. 2026. SWE-MiniSandbox: Container-Free Reinforcement Learning for Building Software Engineering Agents. In *Forty-third International Conference on Machine Learning*.

[263] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Li Aoyan, Lu Chen, Xiaojian Zhong, et al. 2026. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. *Advances in Neural Information Processing Systems* 38 (2026).

[264] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, et al. 2024. SWE-bench-java: A GitHub Issue Resolving Benchmark for Java. *arXiv preprint arXiv:2408.14354* (2024).

[265] Guangtao Zeng, Maohao Shen, Delin Chen, Zhenting Qi, Subhro Das, Dan Gutfreund, David Cox, Gregory Wornell, Wei Lu, Zhang-Wei Hong, et al. 2025. Satori-SWE: Evolutionary Test-Time Scaling for Sample-Efficient Software Engineering. *arXiv preprint arXiv:2505.23604* (2025).

[266] Ji Zeng, Dayuan Fu, Tiantian Mi, Yumin Zhuang, Yaxing Huang, Xuefeng Li, Lyumanshan Ye, Muhang Xie, Qishuo Hua, Zhen Huang, et al. 2026. daVinci-Dev: Agent-Native Mid-Training for Software Engineering. In *Forty-third International Conference on Machine Learning*.

[267] Liang Zeng, Yongcong Li, Yuzhen Xiao, Changshi Li, Chris Yuhao Liu, Rui Yan, Tianwen Wei, Jujie He, Xuchen Song, Yang Liu, et al. 2025. Skywork-SWE: Unveiling Data Scaling Laws for Software Engineering in LLMs. *arXiv preprint arXiv:2506.19290* (2025).

[268] Yucheng Zeng, Shupeng Li, Daxiang Dong, Ruijie Xu, Zimo Chen, Liwei Zheng, Yuxuan Li, Zhe Zhou, Haotian Zhao, Lun Tian, et al. 2026. SWE-Hub: A Unified Production System for Scalable, Executable Software Engineering Tasks. *arXiv preprint arXiv:2603.00575* (2026).

[269] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, et al. 2026. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. *Transactions on Machine Learning Research* (2026). https://openreview.net/forum?id=RY19y2RI1O

[270] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In *The Fourteenth International Conference on Learning Representations*.

[271] Jiwei Zhang, Jianxun Lian, Haiming Qin, Mingyang Zhou, KeZhong Lu, Rui Mao, and Hao Liao. 2025. Hierarchical Reward Modeling for Fault Localization in Large Code Repositories. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 17782–17796.

[272] Kexun Zhang, Weiran Yao, Zuxin Liu, Yihao Feng, Zhiwei Liu, Rithesh Ramapura Narasimha Murthy, Tian Lan, Lei Li, Renze Lou, Jiacheng Xu, et al. 2025. Diversity Empowers Intelligence: Integrating Expertise of Software Engineering Agents. In *The Thirteenth International Conference on Learning Representations*.

[273] Kechi Zhang, Huangzhao Zhang, Ge Li, Jinliang You, Jia Li, Yunfei Zhao, and Zhi Jin. 2026. SEAlign: Alignment Training for Software Engineering Agent. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*.

[274] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, et al. 2026. SWE-bench Goes Live! *Advances in Neural Information Processing Systems* 38 (2026).

[275] Lei Zhang, Jiaxi Yang, Min Yang, Jian Yang, Mouxiang Chen, Jiajun Zhang, Zeyu Cui, Binyuan Hui, and Junyang Lin. 2025. Synthesizing Software Engineering Data in a Test-Driven Manner. In *Forty-second International Conference on Machine Learning*.

[276] Linhao Zhang, Daoguang Zan, Quanshun Yang, Zhirong Huang, Dong Chen, Bo Shen, Tianyu Liu, Yongshun Gong, Huang Pengjie, Xudong Lu, et al. 2025. CodeV: Issue Resolving with Visual Data. In *Findings of the Association for Computational Linguistics: ACL 2025*. 7350–7361.

[277] Quanjun Zhang, Chengyu Gao, Yu Han, Ye Shang, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. SGAgent: Suggestion-Guided LLM-Based Multi-Agent Framework for Repository-Level Software Repair. *ACM Transactions on Software Engineering and Methodology* (2026).

[278] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. 2026. Do Agent Rules Shape or Distort? Guardrails Beat Guidance in Coding Agents. *arXiv e-prints* (2026).

[279] Yikai Zhang, Jiaxin Pei, Kenan Li, Maoquan Wang, Jin Pan, Yu Kang, Shengyu Fu, Elsie Nallipogu, Junjie Hu, Yufan Huang, et al. 2026. SWE-Edit: Rethinking Code Editing for Efficient SWE-agent. *arXiv preprint arXiv:2604.26102* (2026).

[280] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.

[281] Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Zhixiang Wang, Kun Liang, Yang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, et al. 2025. One Tool Is Enough: Reinforcement Learning for Repository-Level LLM Agents. *arXiv preprint arXiv:2512.20957* (2025).

[282] Zejun Zhang, Jian Wang, Qingyun Yang, Yifan Pan, Yi Tang, Yi Li, Zhenchang Xing, Tian Zhang, Xuandong Li, and Guoan Zhang. 2025. A Benchmark for Localizing Code and Non-Code Issues in Software Projects. *arXiv preprint arXiv:2509.25242* (2025).

Just Accepted

[283] Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, et al. 2026. Immersion in the GitHub Universe: Scaling Coding Agents to Mastery. *arXiv preprint arXiv:2602.09892* (2026).
[284] Pengtao Zhao, Boyang Yang, Bach Le, Feng Liu, and Haoye Tian. 2026. Beyond Localization: Recoverable Headroom and Residual Frontier in Repository-Level RAG-APR. *arXiv preprint arXiv:2603.29067* (2026).
[285] Qixing Zhou, JiaCheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, Dandan Tu, and Zhaoxiang Zhang. 2026. FeatureBench: Benchmarking Agentic Coding for Complex Feature Development. In *The Fourteenth International Conference on Learning Representations*.
[286] Xuhui Zhou, Valerie Chen, Zora Zhiruo Wang, Graham Neubig, Maarten Sap, and Xingyao Wang. 2026. ToM-SWE: User Mental Modeling for Software Engineering Agents. In *Forty-third International Conference on Machine Learning*.
[287] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, et al. 2026. SWE Context Bench: A Benchmark for Context Learning in Coding. *arXiv preprint arXiv:2602.08316* (2026).
[288] Yiqi Zhu, Apurva Gandhi, and Graham Neubig. 2025. Training Versatile Coding Agents in Synthetic Environments. *arXiv preprint arXiv:2512.12216* (2025).

Just Accepted

Appendix

A Additional Background of Automatic Issue Resolution

Figure 4 presents the framework of automated issue resolution formed by the five logical phases introduced in Section 2.1. This section describes each phase in detail.

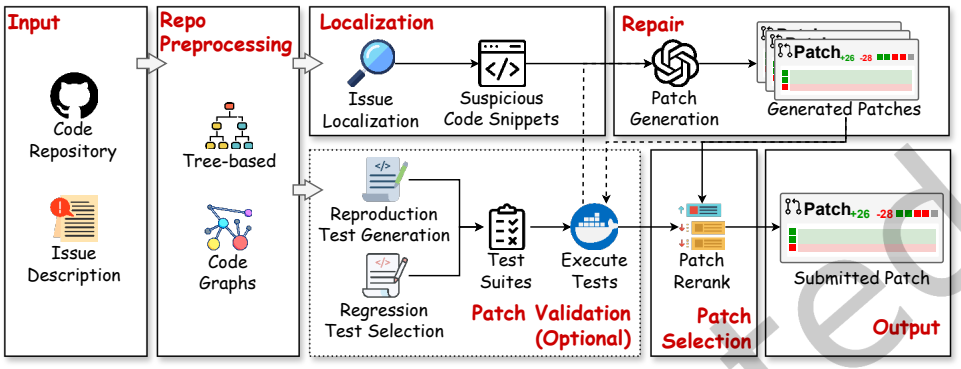


Fig. 4. Overview of the basic framework of automated issue resolution.

Repo Preprocessing. This phase aims to construct an accessible and comprehensible knowledge representation for the code repository. Based on the pre-built knowledge, the LLM or agent can more conveniently and efficiently mine useful information from the repository. For example, constructing a code knowledge graph [32, 251] facilitates the retrieval of inheritance or invocation relationships between classes and functions by the LLM.

Localization. Given the issue description, this phase aims to accurately locate the most relevant code snippets to be edited that are likely responsible for the reported issue. Localization methods may leverage information retrieval [172], static and dynamic program analysis [280], and LLM-based reasoning [236] to identify suspicious files, classes, or lines of code. Effective localization significantly narrows down the search space for patch generation and improves the overall efficiency of the repair process.

Repair. Given the code snippets to be modified and the relevant context, this phase aims to generate patches for issue resolution by leveraging LLMs through prompting techniques. Specifically, the agent produces edits by replacing designated sections of code files [227] and utilizes git tools [252] to generate patches. Typically, multiple candidate patches are generated at this stage to maximize the likelihood of producing a correct fix.

Patch Validation. This phase is adopted by some of the existing techniques. Given the issue description, it aims to generate reproduction test cases or select regression test suites from the repository. Reproduction tests attempt to simulate the issue scenario based on the original issue report [226], while regression tests ensure that the candidate patch does not introduce new errors elsewhere in the codebase [236]. By leveraging testing techniques, this phase filters out some incorrect or harmful patches. Additionally, some techniques run the reproduction or regression tests against candidate patches and use the failing results to guide iterative patch regeneration [227, 252], indicating that validation outcomes may also be fed back to the repair phase.

Patch Selection. Given a set of candidate patches, this phase aims to select the most promising one for submission to resolve the issue. For example, the agent may select a patch based on self-consistency using majority voting [236] or employ a strong reasoning model with LLM-as-a-Judge for evaluation [216]. The selected patch will be applied to the repository as the solution to the issue.

B Literature Search and Selection

B.1 Search Process

The complete search process is illustrated in Figure 5.

Just Accepted

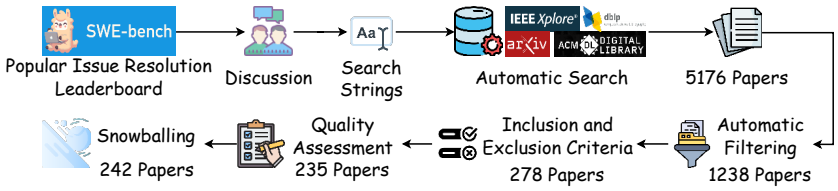


Fig. 5. Overview of the process of paper collection and filtering.

B.1.1 Search Strings. As Jimenez et al. [90] pioneered the issue resolution task, the leaderboard maintained by the SWE-bench [36, 90, 253] team, to some extent, reflects the research progress in this domain. Therefore, we collect 19 papers corresponding to the techniques submitted to the SWE-bench leaderboard as seed papers, manually inspect them, and formulate the search strings for subsequent automated database retrieval through discussion between two of the authors. Specifically, we first extract task-level terms from the seed papers, then expand them with synonyms and subtask terms observed during the manual inspection, and finally group them by software engineering (SE) task scope and artificial intelligence (AI) technical paradigm. The search keywords are summarized in Table 2. Within each domain, keywords are organized into two groups. For SE, the groups follow task granularity: G1 targets end-to-end issue resolution, and G2 targets its key subtasks. For AI, the groups follow technical paradigms: G1 focuses on language models, and G2 on agents. The SWE-bench leaderboard is used only to seed, not to directly populate the paper pool. The subsequent database search and snowballing are not limited to SWE-bench-derived techniques, and therefore also cover studies beyond the SWE-bench family.

Table 2. Keywords related to SE/AI and issue resolution task for automated search.

Group	SE Domain Keywords	AI Domain Keywords
G1	SWE, Issue Solving, Issue Resolution, Issue Fixing, Issue Repair, Program Repair	Large Language Model, LLM, Language Model, LM, Code Large Language Model, Code LLM
G2	Issue Reproduction, Bug Reproduction, Reproduction Test Generation, Issue Localization, Fault Localization, Patch Generation, Patch Rerank, Patch Selection, Patch Validation, Patch Verification	Agent, Software Engineering Agent, SE Agent, SWE Agent

B.1.2 Search Database. We have conducted automated searches in four widely adopted databases, including IEEE Xplore, the ACM Digital Library, arXiv, and DBLP, and selected studies published on or before May 31, 2026. As shown in Table 2, we construct search strings based on the group of keywords. Specifically, within each group, keywords are combined using the "OR" operator, while the SE and AI groups are combined using the "AND" operator, yielding four exact queries. We execute each of these four queries as a full-text search against IEEE Xplore, the ACM Digital Library, and arXiv. DBLP indexes bibliographic metadata rather than full text, so on DBLP the same queries are matched against publication titles. The four query combinations return 1,590 papers for `SE-G1 AND AI-G1`, 997 papers for `SE-G1 AND AI-G2`, 1,339 papers for `SE-G2 AND AI-G1`, and 1,250 papers for `SE-G2 AND AI-G2`, totaling 5,176 raw hits. Given the explosive growth of research on issue resolution, primarily occurring within the past year, we impose no venue restrictions at the query stage and adopt full-text search wherever it is supported to maximize recall. Venue restrictions are instead applied in the subsequent automated filtering, and the inclusion/exclusion and quality assessment stages further control for noise. We consider only publications dated October 2023 or later, as this month marks the initial proposal of the issue resolution task. We then merge duplicate records returned by overlapping queries and databases. Unlike the search, the selection is limited to top-tier venues: during automated filtering, formally published papers are retained only if they appear in top-tier SE and AI conference proceedings and journals, while arXiv preprints are kept for subsequent screening. The considered venues and the number of papers retained from each are TOSEM (196), ICSE (137), FSE (99), ASE (70), ISSTA (49), ICML (43), ACL (36), NeurIPS (31), ICLR (29), TSE (24), MSR (13), EMSE (11), ICSME

Just Accepted

(11), SANER (7), EMNLP (7), ICPC (5), AAAI (5), and NAACL (2), totaling 775 formally published papers alongside 463 arXiv preprints. After this automated filtering, we retain 1,238 papers.

B.2 Inclusion and Exclusion Criteria

Since the automated search only constructs the paper pool through keyword searches, this may result in the inclusion of some papers that are not related to this survey. To ensure relevance and consistency, we define a set of inclusion and exclusion criteria based on prior guidelines [77, 123] to ensure that the papers in the pool match the scope and research questions of our investigation.

Inclusion Criteria. Although our survey centers on the *end-to-end* issue resolution task, subtask-level work on individual stages (e.g., localization, reproduction) is also part of this problem space. Moreover, since benchmarks serve as supporting evaluation infrastructure and empirical studies serve as supporting findings, these papers should be included in our survey scope. Thus, we design inclusion criteria from benchmark, technique, and empirical study dimensions. A paper will be included in this survey if it meets any of the criteria below.

- This paper constructs a dataset or benchmark to evaluate the performance of language models on end-to-end issue resolution tasks.
- This paper constructs a dataset or benchmark to evaluate the performance of large language models (LLMs) on a specific stage of the issue resolution task, such as issue localization or reproduction test generation.
- The paper proposes or improves an approach, method, technique, or tool/framework that leverages LLMs to resolve issues or to address a specific stage of the issue resolution process.
- This paper proposes a specific technique related to LLMs, such as supervised fine-tuning (SFT), for building agents that accomplish issue resolution tasks.
- This paper conducts an empirical or experimental study on issue resolution benchmarks or techniques.

Exclusion Criteria. If a paper meets any of the following criteria, it will be excluded from the scope of our study.

- This paper does not involve any stage of the issue resolution task, such as repository-level code completion.
- This paper does not rely on LLMs for issue resolution.
- This paper primarily evaluates the general performance of LLMs, such as code generation ability or security.
- This paper only mentions LLMs or issue resolution in the related work or future work sections, rather than making them the core focus of the study.
- This paper primarily discusses directions, visions, or ethical considerations, without proposing concrete or implementable technical solutions or findings.

Two authors independently apply the inclusion and exclusion criteria to each paper in the pool. A paper is retained if at least one author votes to include it. After this stage, 278 papers are retained for quality assessment. The two authors reach a Cohen's κ of 0.8487 at this stage, indicating almost perfect agreement. During this stage, the two authors also manually classify each retained paper into three study types according to the inclusion criteria it satisfies: the first two criteria correspond to benchmark papers, the following two to technique papers, and the last one to empirical studies. A paper satisfying criteria of more than one type is assigned to all applicable types. This classification is performed independently, reaching a Cohen's κ of 0.9065, indicating almost perfect agreement, and disagreements are resolved through discussion with a third author. To facilitate the statistical analysis in Appendix B.5, we assign papers belonging to multiple categories one primary category through discussion.

B.3 Quality Assessment

In addition, following prior survey methodologies [77, 86], we define a set of Quality Assessment Criteria (QAC). Since our corpus spans three study types and a single checklist cannot fairly evaluate all of them, we adapt the QAC per category. A set of common criteria applies to every paper, while each set of type-specific criteria applies

Just Accepted

Table 3. Type-adapted Quality Assessment Criteria.

Type	QAC	Type	QAC
All	Is the study related to issue resolution or its sub-tasks?	Technique	Does the study use/train LLMs to build an agentic system?
All	Does the study provide clear and reproducible technical/methodological details?	Technique	Does the study clearly explain how LLMs are used?
All	Does the study present a clear research motivation?	Technique	Does the study specify which LLMs are used?
All	Does the study make a clear contribution to the issue resolution task?	Technique	Does the study provide a clear description of its experimental setup, including experimental environments and dataset information?
All	Are the experimental results and conclusions of the study consistent with its research objectives?	Technique	Does the study include at least one baseline comparison to demonstrate its effectiveness?
Benchmark	Does the study clearly describe its data sources and construction or curation process?	Empirical	Does the study clearly state its research questions?
Benchmark	Does the study report quality-control measures, such as validation, deduplication, or contamination checks?	Empirical	Does the study adopt a clearly described experimental methodology?
Benchmark	Does the study define clear evaluation metrics and protocols?	Empirical	Does the study clearly confirm its empirical findings?

to every paper assigned to that type. Both common criteria and the type-specific criteria are adapted from the QAC of prior surveys [77, 86, 155]. The full set of type-adapted criteria is shown in Table 3. Each paper is scored on the common criteria together with all criteria for its assigned study types. Each QAC item is scored as "yes", "partial", or "no", corresponding to 1, 0.5, and 0, respectively. Following the practice of prior surveys, we exclude a paper if its total score falls below 80% [77, 86] of the maximum attainable over the common criteria together with the type-specific criteria of each type assigned to it, which we refer to as the criteria applicable to that paper.

The same two authors independently applied the type-adapted QAC to all 278 papers retained after the inclusion and exclusion stage and then made binary retain/exclude decisions. They reached a Cohen's κ of 0.9290 on these filtering decisions, indicating almost perfect agreement. When their independent decisions disagreed, a third author adjudicated the case to reach a final decision.

B.4 Snowballing

After quality assessment, we obtain a preliminary paper set. To ensure coverage and reduce the risk of missing relevant studies, we apply a snowballing strategy [123] to expand the set by identifying transitively related work. We conduct both backward and forward snowballing between June 1 and June 10, 2026.[1] Backward snowballing inspects reference lists of collected papers, while forward snowballing uses Google Scholar to identify citing studies. This process yields 19 candidate papers for further screening. After applying the same inclusion, exclusion, and type-adapted quality assessment criteria, 7 papers are retained and added to the 235 papers obtained after quality assessment, resulting in the final set of 242 papers.

B.5 Collection Results and Statistics

As shown in Figure 5, we first discuss and determine the search keywords based on the seed papers collected from the SWE-bench leaderboard [90]. Using these keywords, we construct four exact queries that return 5,176 raw hits. After duplicate removal and automated filtering by publication date and venue, 1,238 papers are retained. After automated filtering, applying the inclusion and exclusion criteria (278 papers), applying the quality assessment criteria (235 papers), and performing the snowballing strategy, we finally identify a collection of 242 papers focused on issue resolution. The papers resulting from each filtering step are released on the artifacts page accompanying this survey.

[1]This period corresponds to the most recent update of our paper collection before finalizing this survey.

Just Accepted

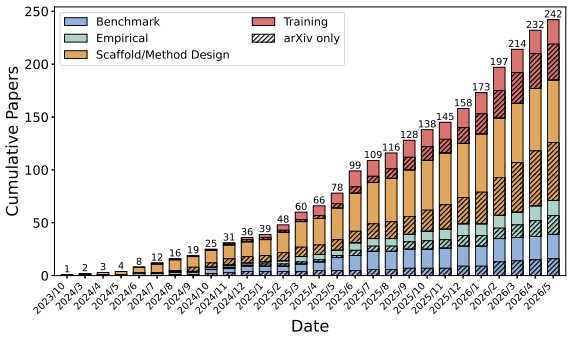


Fig. 6. Trend in the number of papers by month.

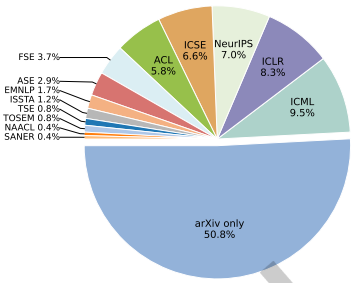


Fig. 7. Distribution of papers across publication venues.

Figure 6 shows the cumulative number of papers published over time as of May 2026. In this figure, each paper is counted under its primary category, and technique papers are further split into scaffold/method design and training studies following the two dimensions of the technique taxonomy in Section 5. We observe a significant gap in this field from October 2023 to May 2024. The number of studies in this area has grown explosively since May 2024, which further highlights the necessity of this survey. Figure 7 presents the publication status of the papers included in this survey. The issue resolution task currently attracts primary attention from the artificial intelligence and software engineering research communities, with 32.6% and 16.5% of the papers published in top-tier venues related to LLM and SE, respectively. Duplicate records are identified and merged by title during data collection. When both an arXiv preprint and a formally published version of the same paper are found, the paper is attributed to its publication venue rather than to arXiv. We further cross-check the arXiv papers against the latest publicly available acceptance lists of the considered conferences, and reassign 25 papers that have been accepted but not yet formally published to their corresponding venues. Notably, 50.8% of the papers have not yet undergone peer review and are available on arXiv. To gauge whether this affects the reliability of our conclusions, we repeat the taxonomy construction procedure of Section 3.3 using only the peer-reviewed subset of the corpus. The resulting top-level taxonomy is unchanged: the same first-level dimensions emerge for benchmarks, techniques, and empirical studies, suggesting that non-peer-reviewed papers have limited influence on our survey's organization. Moreover, our inclusion, exclusion, and type-adapted quality assessment criteria apply uniformly to all papers regardless of venue, which can mitigate the effects.

C Benchmark Details

C.1 End-to-End Benchmark Statistics

Table 4 lists the statistics of end-to-end issue resolution benchmarks.

C.2 Subtask Benchmark Statistics

Table 5 lists the statistics of subtask benchmarks, including issue localization and reproduction test generation.

C.3 Evaluation Metrics

Table 6 maps each execution-based and match-based metric to the representative benchmarks and techniques adopting it. Statistics-based metrics are not listed because they are generic measures applicable to any LLM-based agentic system, whereas we focus on metrics specific to issue resolution.

Just Accepted

Table 4. The statistics and analysis of end-to-end issue resolution benchmarks. System Kind abbreviations: Lib. = library or package; Frm. = framework; Tool = developer, build, testing, or DevOps tool; App. = end-user application; Svc. = service or platform; Sys. = systems or infrastructure software; Mixed OSS = heterogeneous open-source software projects. System Domain terms and abbreviations: AI = Artificial Intelligence; ML = Machine Learning; DS = Data Science; MLOps = ML Operations; CLI = Command-Line Interface; Dev. Tooling = software-development support (e.g., testing, code quality, documentation, and build); Diverse = heterogeneous repositories without a dominant application domain. Language abbreviations: Py = Python; J = Java; JS = JavaScript; TS = TypeScript.

Benchmark	Release Time	#Instances	#Repositories	Language	System Kind	System Domain	Issue Type	Curation	Source
SWE-bench [90]	2023-10	2,294	12	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Bug, Feature	Manual	GitHub
SWE-bench Lite [90]	2023-10	300	12	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Bug	Manual	SWE-bench
SWE-bench Lite-S [236]	2024-07	252	12	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Bug	Manual	SWE-bench
SWE-bench Verified [36]	2024-08	500	12	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Bug	Manual	SWE-bench
SWE-bench-java [264]	2024-08	91	6	J	Lib./Frm./Tool	Data Serialization, Web, DevOps	Bug	Manual	GitHub
SWE-bench Multimodal [253]	2024-10	517	12	JS	Lib./Frm.	Web, Visualization	Bug	Manual	GitHub
SWE-Bench+ [8]	2024-10	548	12	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Bug	Manual	SWE-bench
Visual SWE-bench [276]	2024-12	133	11	Py	Lib./Frm.	DS/ML, Visualization	Bug	Manual	GitHub
SWEE-Bench [214]	2025-03	885	366	Py	Lib.	Diverse (PyPI packages)	Bug	Automatic	GitHub
SWA-Bench [214]	2025-03	535	44	Py	App.	DevOps, Media, Networking, CLI	Bug	Automatic	GitHub
FEA-Bench [116]	2025-03	1,401	83	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Database, Dev. Tooling	Feature	Manual	GitHub
Multi-SWE-bench [263]	2025-04	1,632	39	J, JS, TS, Go, C, Cpp, Rust	Lib./Frm./Tool/Sys.	Web, Data Serialization, Dev. Tooling	Bug, Feature, Performance	Manual	GitHub
LiveSWEBench [134]	2025-04	126	5	Py, JS, TS, J, Cpp	Lib./Frm./Tool/App.	AI/ML, Web, Education, Testing	Bug	Manual	GitHub
SWE-PolyBench [169]	2025-04	2,110	21	Py, J, JS, TS	Lib./Frm./Tool/App.	Web, AI/ML, Cloud/Middleware, Dev. Tooling	Bug, Feature, Refactoring	Automatic	GitHub
OmniGIRL [65]	2025-05	959	15	Py, J, JS, TS	Lib./Frm./Tool	Code Quality, Web, Time Utilities, Networking, Statistics, Testing, Cryptography	Bug	Manual	GitHub
SWE-bench Multilingual [95]	2025-05	300	42	J, JS, TS, Go, C, Cpp, PHP, Ruby, Rust	Lib./Frm./Tool/Sys.	Web, Data Storage, Dev. Tooling	Bug, Feature	Manual	GitHub
SwingArena [245]	2025-05	400	N/A	Rust, Py, Go, Cpp	Mixed OSS	Diverse	Bug	Manual	GitHub
SWE-rebench [12]	2025-05	21,000+	3,468	Py	Mixed OSS	Diverse	Bug	Automatic	GitHub
SWE-bench-Live [274]	2025-05	1,319	93	Py	Mixed OSS	AI/ML, DevOps, Web, Scientific Computing, Database	Bug	Automatic	GitHub
GSO [185]	2025-05	102	10	Py, C, Cpp, Cython, Rust	Lib./Frm./Sys.	Scientific Computing, Data Analysis, Image Processing, Web, Data Validation, ML	Performance	Manual	GitHub
SWE-MERA [161]	2025-07	300	200	Py	Mixed OSS	Diverse	Bug	Automatic	GitHub
SWE-Perf [75]	2025-07	140	9	Py	Lib./Frm./Tool	DS/ML, Scientific Computing	Performance	Automatic	SWE-bench
Doc2Feat-bench [40, 88]	2025-07	634	10	Py	Lib./Frm./Tool	DS/ML, Scientific Computing, Web, Dev. Tooling	Feature	Manual	GitHub
SWE-Bench Pro [41]	2025-09	1,865	41	Py, Go, JS, TS	App./Svc./Tool	Consumer Apps, B2B/Enterprise, Dev. Tooling	Bug, Feature	Manual	GitHub/Proprietary
SWE-fficiency [138]	2025-11	498	9	Py	Lib./Frm.	DS/ML, High-Performance Computing, Scientific Computing	Performance	Manual	GitHub
SWE-Bench++ [221]	2025-12	11,133	3,971	Py, J, JS, TS, Go, Rust, C, Cpp, Ruby, PHP, C#	Mixed OSS	Diverse	Bug, Feature	Automatic	GitHub
SWE-EVO [209]	2025-12	48	7	Py	Lib./Frm./Tool	DS/ML, Package Management, HTTP, Data Processing	Bug, Feature	Manual	GitHub
SWE-Refactor [247]	2026-02	1,099	18	J	Lib./Frm./Tool	Utilities, Testing, Code Quality, Web Middleware	Refactoring	Automatic	GitHub
FeatureBench [285]	2026-02	200	24	Py	Lib./Frm./Tool	DS/ML, MLOps, Web, Scientific Computing, Dev. Tooling	Feature	Automatic	GitHub
Rust-SWE-bench [239]	2026-02	500	34	Rust	Lib./Frm./Tool/Sys.	Systems Infrastructure, Web, CLI Utilities	Bug, Feature, Refactoring	Manual	GitHub
SWE-Bench Mobile [210]	2026-02	50	1	Swift, Objective-C	App.	Mobile (iOS)	Feature	Manual	Proprietary
SWE-CI [25]	2026-03	100	68	Py	Mixed OSS	Diverse	Bug, Feature, Maintenance	Automatic	GitHub
SWE-STEPS [183]	2026-04	168	6	Py	Lib./Frm./Tool	Build Automation, AI, Scientific Computing, Database, Testing	Bug, Feature, Maintenance	Automatic	GitHub
SWE-Chain [105]	2026-05	155	9	Py	Lib./Frm./Tool	Web, Testing, DS, HTTP, Package Management	Bug, Feature	Manual	PyPI

Just Accepted

Table 5. The statistics and analysis of issue resolution subtask benchmarks.

Benchmark	Release Time	#Instances	#Repositories	Domain	Issue Type	Curation	Source
Localization							
LocBench [32]	2025-03	660	N/A	DS/ML, Web, Scientific Computing, Tools	Bug, Feature, Performance, Security	Manual	GitHub
MULocBench [282]	2025-09	1,100	46	DS/ML, Web, Tools, Systems	Bug, Feature	Manual	GitHub
ContextBench [112]	2026-02	1,136	66	DS/ML, Web, Tools, Systems	Bug, Feature	Manual	GitHub
SWE Context Bench [287]	2026-02	399	12	DS/ML, Web, Scientific Computing, Tools	Bug	Manual	SWE-bench
Reproduction							
SWT-Bench [152]	2024-06	1,978	12	DS/ML, Web, Scientific Computing, Tools	Bug	Automatic	SWE-bench
TestGenEval [82]	2024-10	1,210	11	DS/ML, Web, Scientific Computing, Tools	Test	Automatic	SWE-bench
TDD-Bench [7]	2024-11	449	12	DS/ML, Web, Scientific Computing, Tools	Bug	Manual	SWE-bench

Table 6. Categorization of evaluation metrics for issue resolution. The Usage column lists representative benchmarks and techniques that adopt each metric.

Category	Metric	Scope	Usage
Execution-based	Applied%	End-to-End	[12, 40, 41, 75, 88, 90, 116, 142, 152, 154, 157, 169, 205, 209, 210, 214, 216, 226, 251, 268, 274]
	Resolved% / Pass@k	End-to-End	[3, 41, 46, 61, 82, 90, 97, 116, 131, 159, 190, 209, 210, 236, 252, 253, 263–265, 268, 274, 280]
	Reproduction Success Rate	Reproduction	[4–7, 40, 49, 88, 96, 101, 122, 150–152, 154, 190, 194, 219, 226, 239, 283]
	Delta Change Coverage	Reproduction	[4, 5, 7, 49, 96, 115, 152, 154, 190, 219, 226]
	Speedup	End-to-End	[75, 138, 185]
	Opt@k	End-to-End	[75, 138, 185]
Match-based	File Matched Rate	Localization	[90, 169, 236, 261]
	Function Matched Rate	Localization	[90, 169, 236, 261]
	Top-k	Localization	[32, 48, 70, 89, 112, 130, 131, 136, 145, 147, 171, 172, 182, 217, 219, 243, 246, 271, 282, 287]
	MAP	Localization	[22, 23, 32, 70, 89, 112, 130, 145, 219, 271, 282, 287]
	MRR	Localization	[22, 23, 32, 48, 70, 89, 112, 130, 145, 147, 182, 196, 219, 243, 271, 282, 287]
	Precision, Recall, and F1	Localization	[32, 70, 90, 112, 116, 130, 132, 145, 169, 175, 181, 188, 198, 205, 207, 243, 244, 246, 271, 281, 282]

C.3.1 Execution-based Metrics. In general, benchmarks related to code generation often rely on test suites for evaluation. As a challenging code generation task, issue resolution depends on two types of test suites for end-to-end evaluation. The first type consists of test cases whose outcomes change from "FAIL" to "PASS" after applying the golden patch, i.e., the tests that verify whether the issue is resolved (denoted as $F \rightarrow P$). The second type consists of test cases whose outcomes remain "PASS" both before and after applying the golden patch, i.e., regression tests (denoted as $P \rightarrow P$). Based on the execution results of these two types of test suites, existing work mainly uses the following metrics.

- **Applied%:** The proportion of candidate patches that can be successfully applied to the repository and initiate the execution of the test suites.
- **Resolved%:** The proportion of candidate patches that pass all $F \rightarrow P$ and $P \rightarrow P$ tests.
- **Pass@k (a.k.a. Resolved@k):** The proportion of cases in which at least one of the k patches generated by the agentic system passes all $F \rightarrow P$ and $P \rightarrow P$ tests.
- **Reproduction Success Rate:** In the issue reproduction stage, the generated reproduction test qualifies as an $F \rightarrow P$ test and, after applying the generated test patch, passes all $P \rightarrow P$ tests.
- **Delta Change Coverage:** It evaluates how well a generated reproduction test covers the code changes introduced by the ground-truth patch [219]. Formally, for instance i with modified patch-line set M_i and covered patch-line set $C_i(\hat{t}_i)$ under generated test $\hat{t}_i$, $\Delta C_i = |M_i \cap C_i(\hat{t}_i)|/|M_i|$.

Just Accepted

- **Speedup:** In performance optimization tasks, the ratio of the execution time of the target tests before applying the candidate patch to that after applying it, measuring the runtime improvement delivered by the patch [75, 185].
- **Opt@k:** The proportion of instances in which at least one of the k candidate patches passes all correctness tests and achieves at least a predefined fraction of the speedup delivered by the ground-truth patch [185].

C.3.2 Match-based Metrics. Match-based metrics are mainly used to evaluate agentic system performance in the localization stage. These metrics compare predicted locations against reference locations derived from the developer patch. Since the developer patch is not the only possible fix, the reference locations may miss code involved in alternative correct fixes, so matching results measure agreement with the developer fix rather than complete localization correctness. However, due to the autonomy of agent decision-making, not all agentic systems explicitly output intermediate results of the localization stage. Therefore, we discuss the following metrics from two perspectives: the generated patches (File/Function Matched Rate) and the intermediate localization results (Top-k, MAP, MRR, Precision, Recall, and F1).

- **File Matched Rate:** The proportion of instances where the set of files modified by the candidate patch intersects with the reference file set. Formally, for instance i with predicted file set $\hat{F}_i$ and reference set F_i, it counts as correct if $\hat{F}_i \cap F_i \neq \emptyset$.
- **Function Matched Rate:** The proportion of instances where the set of modified functions or methods in the candidate patch intersects with the reference function set, that is $\hat{G}_i \cap G_i \neq \emptyset$ for instance i.
- **Top-k (a.k.a. Hit@k):** Given a ranked list of candidate locations produced during localization, the proportion of instances where at least one reference location appears within the top k positions. When there are multiple reference locations, success is recorded if any of them is within rank k.
- **Mean Average Precision (MAP):** It assesses ranking quality by averaging the precision values computed at the ranks of all correctly identified buggy elements within the recommendation list. Formally, MAP is defined as follows: $MAP = \frac{1}{n}\sum_{j=1}^{n} \text{AvgP}_j,\ \text{AvgP}_j = \frac{1}{|K_j|}\sum_{k\in K_j} \text{Prec@}k,\ \text{Prec@}k = \frac{1}{k}\sum_{i=1}^{k} \text{IsRelevant}(i)$ where $|K_j|$ is the number of buggy elements for the j-th bug, and $\text{IsRelevant}(i)$ returns 1 if the i-th element is buggy, and 0 otherwise. K_j denotes the ranks of all buggy elements associated with the j-th bug.
- **Mean Reciprocal Rank (MRR):** It assesses ranking quality by computing the reciprocal of the position of the earliest correctly identified buggy element in the recommendation list. Formally, MRR is defined as: $MRR = \frac{1}{n}\sum_{j=1}^{n} \frac{1}{\text{rank}_j}$ where rank_j is the ranking position of the first buggy method that was modified to resolve the j-th bug in the recommendation list.
- **Precision, Recall, and F1:** They evaluate the overlap between predicted and reference buggy locations from complementary perspectives [130]. Formally, for instance i with predicted location set $\hat{L}_i$ and reference location set L_i, $\text{Precision} = |\hat{L}_i \cap L_i|/|\hat{L}_i|$, $\text{Recall} = |\hat{L}_i \cap L_i|/|L_i|$, and $F1 = 2 \cdot \text{Precision} \cdot \text{Recall}/(\text{Precision} + \text{Recall})$.

C.3.3 Statistics-based Metrics. Agentic systems typically rely on LLMs to perform multi-turn reasoning. Therefore, besides effectiveness, it is also necessary to evaluate system efficiency. This is measured by tracking the average number of consumed input and output tokens (#Token), dollar cost ($Cost), and time consumed (Time) per issue during system execution.

C.4 Benchmark Construction Details

This section details the construction processes and representative benchmarks summarized in Section 4.1.

C.4.1 Manually Constructed Benchmarks. The manual construction of issue resolution-related benchmarks typically involves five phases, as illustrated in Figure 8. First, well-maintained repositories are carefully selected from code hosting platforms such as GitHub, and all pull request (PR) data is collected. Second, the raw data is

Just Accepted

processed with handcrafted parsing rules to filter out instances irrelevant to the target tasks. Third, the execution environments of the selected projects are manually configured by consulting the corresponding documentation to ensure their executability. Fourth, all test cases are executed before and after applying the patches associated with each PR, and instances that demonstrate consistent changes in test outcomes are preserved as valid benchmark data. Finally, developers may optionally annotate factors such as the clarity of problem descriptions and the coverage of test cases, and use these criteria to filter out a high-quality subset of data for evaluation.

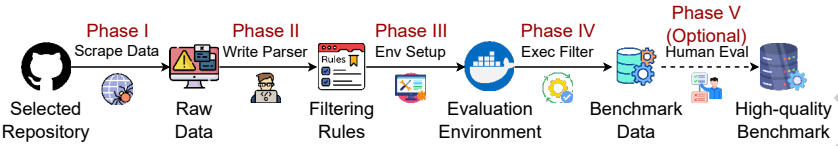


Fig. 8. A typical pipeline of manual construction of issue resolution-related benchmarks.

In the early stages of the issue resolution task, nearly all benchmarks relied on manual construction. SWE-bench [90] is the first benchmark that implements the construction pipeline illustrated in Figure 8. It collects approximately 90,000 PRs from 12 carefully selected repositories, which, after rule-based filtering and execution-based validation, yield 2,294 instances for evaluation. Subsequent efforts generally take SWE-bench as a reference paradigm, enhancing and extending its methodology (as discussed in Section 4.2) to incorporate multimodal [65, 253, 276], multilingual [95, 263, 264], and diverse code repositories [41, 134, 169, 274], thereby broadening the scope of issue resolution evaluation.

C.4.2 Automatically Constructed Benchmarks. The automatic construction of issue resolution benchmarks aims to rapidly collect large-scale issue resolution data to compensate for the limited timeliness of static benchmarks. In particular, existing methods primarily target environment setup, which is the most labor-intensive process. For example, SWEE/SWA-Bench [214] employs a SetupAgent that automates environment setup through three phases—Command Extraction, Iterative Testing and Improvement, and Validation—constructing installation commands, testing commands, and result parsers. SWE-bench-Live [274] adopts an end-to-end agentic workflow, RepoLaunch, which reads documentation, iteratively executes and debugs commands, and refines strategies based on feedback to complete environment setup. Similarly, SWE-rebench [12] and SWE-CI [25] build automated pipelines that continuously collect and validate tasks from GitHub repositories and continuous-integration histories, respectively. Such automated construction methods can also be used to collect training datasets to train LLMs, which is further discussed in Section 5.2.1.C.

C.5 Benchmark Extension Details

This section details representative benchmarks along the five extension directions summarized in Section 4.2.2. Statistics for end-to-end and subtask benchmarks are provided in Tables 4 and 5, respectively.

C.5.1 Multimodal. Yang et al. [253] select four categories of tasks in JavaScript repositories, including diagramming, interactive mapping, syntax highlighting, and web frameworks, and construct SWE-bench Multimodal following the construction pipeline of SWE-bench [90]. Zhang et al. [276] and Guo et al. [65] select tasks with image-based descriptions from SWE-bench and GitHub to construct Visual SWE-bench and OmniGIRL, respectively.

C.5.2 Multilingual. Zan et al. [264] first migrate SWE-bench to the Java language, after which the ByteDance team further extends it to ten additional programming languages, constructing Multi-SWE-bench [263]. Similarly, Rashid et al. [169], Guo et al. [65], and Khandpur et al. [95] also construct multilingual issue resolution benchmarks named SWE-PolyBench [169], OmniGIRL [65], and SWE-bench Multilingual [95], with the specific programming

Just Accepted

languages involved summarized in Table 4. Recent efforts further broaden language and platform coverage, including SwingArena [245], Rust-SWE-bench [239], SWE-Bench Mobile [210], and SWE-Bench++ [221].

C.5.3 Timeliness. LiveSWEBench [134] and SWE-MERA [161] use manually designed pipelines to automatically collect the latest issue data from target repositories. In contrast, SWE-bench-Live [274] designs a fully agentic workflow to mitigate performance saturation caused by benchmark obsolescence.

C.5.4 Other Issue Types. Researchers now extend benchmarks to four categories of non-bug-fixing tasks. For feature addition tasks [40, 88, 116, 285] and performance optimization tasks [75, 138, 185], benchmarks evaluate agents' ability to implement new functionality and improve program efficiency, respectively. For code refactoring tasks, SWE-Refactor [247] evaluates repository-level tasks from real-world Java pull requests, requiring agents to preserve program behavior while improving code structure. Long-term software evolution tasks [105, 183, 209] further evaluate agents across sequences of interdependent repository changes.

C.5.5 Subtasks. For issue localization, LocBench [32] collects four categories of issue data, including bug reports, feature requests, security issues, and performance issues, and applies traditional fault localization evaluation methods [112, 287] to assess the fine-grained performance of LLMs in localizing different types of issues. MULocBench [282] extends the localization problem beyond source code to include other non-code files such as commits, comments, configurations, and documentation, offering greater diversity in terms of issue types, root causes, localization scopes, and file categories. For issue reproduction, SWT-Bench [152], TestGenEval [82], and TDD-Bench [7] all derive instances from SWE-bench and, through different filtering criteria, evaluate the ability of LLMs to reproduce issues in SWE-bench.

D Techniques

D.1 End-to-End Scaffolds Overview.

Table 7 presents and analyzes the chronological development of existing end-to-end scaffolds and the performance-cost trade-off.

D.2 Repository Representation Methods

This section details representative methods of the two repository representation types summarized in Section 5.1.1.A.

A.1 Tree-based Representation. Tree-based methods use static parsers to organize repository files, classes, and functions into lightweight hierarchies that fit within the LLM context window. For example, Agentless [236] statically parses the repository into a repository-level tree of files and a file-level tree of classes and functions, illustrated in Figure 9, so that the LLM sees the repository skeleton without reading any function body. Because they are simple and inexpensive to construct, similar representations are widely used by pipeline-based scaffolds and related systems [80, 110, 151, 157, 188, 241, 276].

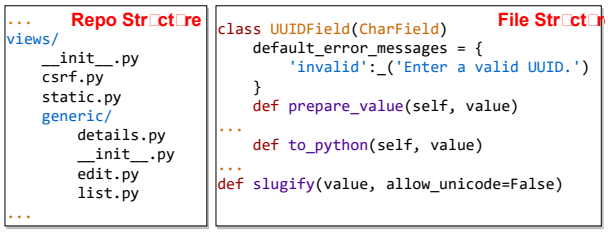


Fig. 9. Tree-based representation of repository and code files.

Just Accepted

Table 7. End-to-end scaffolds overview. MA = Multi-Agent; MCTS = Monte Carlo tree search; Valid. = Validation; Reprod. = Reproduction; Regress. = Regression; Sel. = Selection. Cost is the average USD expense per issue. Resolved% and Cost are colored by the benchmark on which they are reported: SWE-bench Verified, SWE-bench Lite, and other benchmarks.

Method	Release Time	MA	Repo Preprocessing	Localization	Patch Valid.		Patch Sel.	Extra Characteristic	Backbone	Resolved%	Cost
			Representation		Reprod.	Regress.	Rerank				
Agent-based Methods											
MAGIS [205]	2024-03	✓	✗	BM25	✗	✗	✗	Code Review	GPT-4	-	-
AutoCodeRover [280]	2024-04	✓	✗	Navigation/Spectrum	✗	✗	✗	-	GPT-4	19.0	$0.43
SWE-agent [252]	2024-05	✗	✗	Navigation	✓	✗	✗	-	GPT-4 Turbo	18.0	$1.67
CodeR [24]	2024-06	✓	✗	BM25/Spectrum	✓	✗	✗	Code Task Graph	GPT-4 Turbo	28.3	$3.09
MASAI [216]	2024-06	✓	✗	Navigation	✓	✗	✓	Test Template Generation	GPT-4o	28.3	$1.96
Alibaba LingmaAgent [142]	2024-06	✓	Code Graph [142]	BM25/Graph	✗	✗	✗	MCTS	Claude 3.5 Sonnet	38.3	$2.18
OpenHands CodeAct [227]	2024-07	✗	✗	Navigation	✓	✓	✗	General SE Agent	Claude 3.5 Sonnet	26.0	$1.10
CodeXGraph [131]	2024-08	✗	Code Graph [131]	Graph	✗	✗	✗	-	GPT-4o	23.0	-
SpecRover [175]	2024-08	✓	✗	Navigation	✓	✓	✓	Intent Infer	Claude 3.5 Sonnet	31.0	$0.65
MarsCode Agent [132]	2024-09	✓	Code Graph [132]	Navigation	✓	✗	✓	Language Server	GPT-4o	34.0	-
HyperAgent [165]	2024-09	✓	✗	Navigation	✗	✗	✗	-	GPT-4o	31.4	$2.01
SuperCoder [59]	2024-09	✓	✗	Embedding	✗	✗	✗	-	GPT-4/Claude 3.5 Sonnet	34.0	-
SWE-Search [11]	2024-10	✗	✗	Navigation	✗	✗	✗	MCTS	GPT-4o	31.0	-
Infant Agent [107]	2024-11	✓	✗	Navigation	✗	✗	✗	-	GPT-4o	30.0	-
Learn-by-interact [193]	2025-01	✗	✗	Navigation	✗	✗	✗	Synthesized Data	Claude 3.5 Sonnet	60.0	-
DARS [3]	2025-03	✗	Code Graph [157]	Navigation	✓	✗	✓	-	Claude 3.5 Sonnet	47.0	$12.24
InfantAgent-Next [106]	2025-05	✓	✗	Navigation	✗	✗	✗	Multimodal Support	GPT-4o + DeepSeek-V3	31.7	-
OpenHands-Versa [191]	2025-06	✗	✗	Navigation	✓	✓	✗	Multimodal General Agent	Claude 4 Sonnet	34.4	$1.79
EXPEREPAIR [151]	2025-06	✓	Tree-based [236]	Navigation	✓	✗	✓	Dual Memory/Experience	Claude 4 Sonnet + o4-mini	74.6	$1.91
Agent KB [200]	2025-07	✗	✗	Navigation	✓	✓	✗	Experience	Claude 3.7 Sonnet	48.3	-
Prometheus [31]	2025-07	✓	Code Graph [31]	Navigation/Graph	✓	✗	✗	-	DeepSeek-V3	28.7	$0.23
SWE-Exp [27]	2025-07	✓	✗	Navigation	✗	✗	✗	Experience	DeepSeek-V3	42.0	$0.13
SWE-Debate [109]	2025-07	✓	Code Graph [109]	Navigation/Graph	✗	✗	✗	Debate	DeepSeek-V3	41.4	-
TRAE [56, 211]	2025-07	✓	✗	Navigation	✓	✓	✓	-	Claude 3.7 Sonnet	66.4	-
SE-Agent [68]	2025-08	✗	✗	Navigation	✗	✗	✗	Self-Evolution	Claude 3.7 Sonnet	61.2	-
Lita [39]	2025-09	✗	✗	Navigation	✓	✗	✗	Minimalist Design	Claude 4 Opus	62.6	-
Lingxi [255]	2025-10	✓	✗	Navigation	✓	✓	✗	Procedural Knowledge	Claude 4 Sonnet	74.6	-
InfCode [114]	2025-11	✓	✗	Navigation	✓	✗	✓	Adversarial Refine	DeepSeek-V3	40.3	$0.26
Confucius Code Agent [232]	2025-12	✓	✗	Navigation	✓	✗	✗	Hierarchical Memory	Claude 4 Sonnet	74.6	-
Agyn [15]	2026-02	✓	✗	Navigation	✗	✓	✗	Role-Based Team	GPT-5 + GPT-5-Codex	72.2	-
Debug2Fix [57]	2026-02	✓	✗	Navigation	✗	✗	✗	Interactive Debugging	Claude 4.5 Sonnet	40.4	-
SGAgent [277]	2026-02	✓	Code Graph [277]	Graph	✓	✓	✓	Locate-Suggest-Fix	Claude 3.5 Sonnet	51.3	$1.48
SWE-Adept [74]	2026-03	✓	Tree-based [74]	Navigation	✓	✓	✓	Hypothesis-Driven	Claude 4.5 Sonnet	71.3	-
iSWE Agent [54]	2026-03	✓	✗	Navigation	✗	✗	✗	Java-Specialized	Claude 4.5 Sonnet	33.6	$1.86
REAgent [103]	2026-04	✓	✗	Navigation	✓	✓	✓	Requirements-Driven	DeepSeek-V3.2	46.0	-
Agent-CoEvo [115]	2026-04	✓	✗	Navigation	✓	✗	✓	Coevolutionary Search	DeepSeek-V3	41.3	$1.11
Pipeline-based Methods											
Agentless [236]	2024-07	✗	Tree-based [236]	Navigation/Embedding	✓	✓	✓	-	GPT-4o	32.0	$0.70
RepoGraph [157]	2024-10	✗	Code Graph [157] Tree-based [236]	Navigation	✗	✗	✓	Code Relation Graph	GPT-4o	29.7	$0.39
SWESynInfer [140]	2024-11	✗	✗	Navigation	✗	✗	✗	-	Lingma SWE-GPT 72B [140]	30.2	-
SWE-Fixer [241]	2025-01	✗	Tree-based [236]	BM25/Navigation	✗	✗	✗	-	SWE-Fixer-72B [241]	32.8	-
Agentless-Mini [233]	2025-02	✗	Tree-based [236]	Navigation	✓	✓	✓	-	Llama3-SWE-RL-70B [233]	41.0	-
CodeV [276]	2025-02	✗	Tree-based [236]	Navigation	✗	✗	✓	Multimodal Support	Qwen2-VL + Qwen2.5-Coder-32B	12.8	-
CodeMonkeys [46]	2025-02	✗	✗	Navigation	✓	✗	✓	Multi-Model	Claude 3.5 Sonnet	57.4	$4.58
PatchPilot [110]	2025-02	✗	Tree-based [236]	Navigation/Embedding	✓	✓	✓	Iterative Refinement	Claude 3.5 Sonnet	53.6	$0.99
KGCompass [251]	2025-03	✗	✗	Graph	✓	✗	✓	PR Knowledge Graph	Claude 4 Sonnet	58.3	$0.20
Jiang et al. [87]	2025-05	✗	✗	-	✗	✗	✗	Direct Solve	Gemini 2.5 Pro	50.8	-
CGM [204]	2025-05	✗	Code Graph [204]	Embedding	✗	✗	✗	GraphRAG	Qwen2.5-72B-Instruct [204]	50.4	-
GUIRepair [80]	2025-06	✗	Tree-based [236]	Navigation/Embedding	✓	✗	✓	Render Check	o4-mini	33.9	$0.36
SemAgent [158]	2025-06	✗	✗	Spectrum	✓	✗	✓	Semantic Understanding	Claude 3.7 Sonnet	44.7	$6.90
Nemotron-Cortexa [188]	2025-06	✗	Tree-based [236]	Navigation/Embedding	✓	✗	✓	Code Embedding Model	Claude 3.5 Sonnet	52.6	$0.51
SynFix [199]	2025-07	✗	Code Graph [199]	Graph/Embedding	✓	✓	✗	Code Relation Graph	GPT-4o	52.3	$0.56
SIADAFIX [20]	2025-10	✓	✗	Navigation	✓	✓	✓	Adaptive Workflow	Claude 4 Sonnet	60.7	-
TDFlow [72]	2025-10	✗	✗	Navigation	✓	✓	✓	Test-Driven Workflow	GPT-5 + Claude 4 Sonnet	68.0	$4.12
Think-Search-Patch [243]	2025-11	✗	✗	Embedding	✗	✗	✗	Two-Stage Training	Qwen2.5-Coder-14B [243]	8.3	-
SVRepair [201]	2026-02	✗	✗	Navigation	✗	✓	✗	Multimodal Visual	SVR-8B [201] + o3	36.5	-
RepoRepair [160]	2026-03	✗	✗	Embedding	✗	✓	✓	Documentation-Enhanced	Claude 4 Sonnet + DeepSeek-V3	45.7	$0.44
RAIM [124]	2026-03	✗	✗	Navigation	✗	✓	✓	Feature Addition	Gemini 2.5 Pro	39.5	-
ARISE [181]	2026-05	✗	Code Graph [181]	Graph	✗	✗	✗	Graph Toolset	Qwen3.6-35B-A3B	34.0	-

Just Accepted

A.2 Code Graph. Code-graph methods represent deeper repository semantics through relations such as function calls, inheritance, imports, references, and data flows. Existing systems differ mainly in graph granularity and access mechanism: they construct function-level call graphs [142, 199, 204], line-level definition-reference graphs [3, 157], or richer dependency graphs over code and documentation [31, 109, 131, 132]. Some supply graph-derived context directly to the LLM, whereas CodeXGraph [131] and Prometheus [31] expose graph-database query interfaces. For example, CodeXGraph [131] persists the extracted nodes and edges in a graph database and lets the LLM retrieve context by generating Cypher queries over it.

D.3 Localization Methods in End-to-End Scaffolds

This section details representative methods of the five localization types summarized in Section 5.1.1.B.

B.1 BM25. BM25 [173] performs lexical retrieval and is well suited to long issue descriptions and code documents [90]. Existing scaffolds use it either for initial suspicious-file retrieval [205, 241] or as a signal combined with search and reasoning, such as MCTS in Alibaba LingmaAgent [142] and spectrum scores in CodeR [24]. For example, SWE-Fixer [241] scores every repository file against the issue description as a query and forwards only the top-ranked files to its repair model.

B.2 Spectrum-based Methods. Spectrum-based fault localization (SBFL) contrasts coverage from passing and failing tests and ranks program elements with suspiciousness metrics such as Tarantula [91] and Ochiai [1]. Issue-resolution systems use these rankings directly as fault-revealing context [280], combine them with lexical retrieval [24], or derive semantic cues from bug reports [158]. For example, AutoCodeRover [280] runs the test suite with coverage, ranks methods by suspiciousness, and passes the top-ranked ones to the agent as fault-revealing context. Because reproduction tests are often unavailable or difficult to generate [152], the adoption of SBFL remains limited.

B.3 Embedding-based Methods. Embedding-based methods project issue descriptions and code segments into a shared semantic space and retrieve code by vector similarity [157, 236]. This supports coarse-grained localization in scaffolds such as Agentless, SuperCoder, PatchPilot, GUIRepair, and SynFix [59, 80, 110, 199, 236]. For example, Agentless [236] splits repository files into code segments and ranks them by the cosine similarity between their embeddings and that of the issue description. Recent variants enrich vector retrieval with code graphs [204] or fine-tune the embedding model for repository-level file retrieval [188].

B.4 Navigation-based Methods. Navigation-based methods let the LLM inspect a repository through either specialized search operations or a general shell interface. Agent-based scaffolds dynamically select fine-grained tools [24, 175, 280] or general bash commands [3, 11, 27, 31, 56, 68, 106, 107, 109, 132, 151, 165, 191, 193, 200, 216, 227, 252], whereas pipeline-based scaffolds constrain navigation to predefined file-to-function hierarchies [80, 110, 140, 157, 188, 233, 236, 241, 276]. For example, SWE-agent [252] repeats an observe–act loop over its agent–computer interface, deciding each search or edit operation from the returned repository state, whereas Agentless [236] follows a fixed cascade that narrows from the structure tree to files, then to classes and functions, and finally to edit locations. Other designs either aggregate LLM-ranked files [46] or use long-context models to inspect an entire repository [87].

B.5 Graph-based Methods. Graph-based localization retrieves issue-relevant context from the repository graphs introduced in Appendix D.2. Most methods reason over graph neighborhoods or paths: Alibaba LingmaAgent constrains exploration with MCTS [142], KGCompass combines graph traversal with LLM-proposed candidates [251], SynFix combines embedding retrieval with one-hop expansion [199], and SWE-Debate compares multiple fault-propagation paths [109]. For example, SWE-Debate [109] enumerates candidate fault-propagation paths from the entities named in the issue and lets multiple agents debate them, adopting the surviving path as the localization result. A second design exposes graph-database queries, allowing the LLM to retrieve context through generated Cypher commands [31, 131]. These methods capture structural dependencies beyond lexical similarity but depend on the quality and accessibility of the underlying graph.

D.4 Training Dataset Statistics

Table 8 lists the statistics of existing training datasets for training the issue resolution model.

[2]Skywork-SWE dataset has not been open-sourced yet.

Just Accepted

Table 8. Comparison of existing training datasets for software engineering agents. For Env. Size, "-" indicates the dataset is not executable. "Not Released" indicates the dataset authors do not release the official snapshot. "Not Stat." indicates the dataset authors do not report the size of the snapshot.

Dataset	Release Time	#Instances	Executable?	Env. Size	Type
SWE-bench(Train) [90]	2023-10	19,008	✗	-	Real
R2E [83]	2024-07	246	✓	270 GB	Synthetic
SWE-Gym [159]	2024-12	2,438	✓	6 TB	Real
SWE-bench-extra [14]	2024-12	6,411	✓	Not Released	Real
SWE-Fixer [241]	2025-01	110k	✗	-	Real
R2E-Gym(Subset) [84]	2025-04	4,578	✓	4 TB	Synthetic
SWE-Synth [164]	2025-04	9,459	✓	Not Released	Synthetic
SWE-smith [254]	2025-04	50,137	✓	295 GB	Synthetic
Multi-SWE-RL [263]	2025-04	4,723	✓	Not Stat.	Real
SWE-rebench [12]	2025-05	20k	✓	Not Stat.	Real
SWE-Dev(a) [45]	2025-06	14.5k	✓	100 GB	Real
SWE-Dev(b) [218]	2025-06	26k	✓	Not Released	Synthetic
Skywork-SWE[2] [267]	2025-06	10k	✓	Not Released	Real
SWE-Flow [275]	2025-06	16k	✓	Not Stat.	Synthetic
SWE-Mirror [220]	2025-09	60k	✓	100 GB	Synthetic
SWE-Universe [26]	2026-02	807,693	✓	Not Released	Real
Scale-SWE-Data [283]	2026-02	100k	✓	Not Stat.	Real
Hybrid-Gym [242]	2026-02	4,470	✓	Not Stat.	Synthetic
SWE-rebench V2 [13]	2026-02	32,079	✓	26.36 TB	Real
SWE-Next [119]	2026-03	2,308	✓	639 GB	Real
STITCH [208]	2026-04	2,067	✓	Not Stat.	Real

D.5 Training Method Statistics

Table 9 summarizes representative models, their training paradigms, and their performance on SWE-bench Verified.

Just Accepted

Table 9. Training methods and performance statistics of domain-specific models for the issue resolution task. Resolved% represents the reported score of each model on SWE-bench Verified when used with its corresponding scaffold. "Process" indicates process-oriented methods; "Distillation" indicates teacher-model distillation methods; "PRM" indicates process reward model; "ORM" indicates outcome reward model.

Method/Model	Release Time	Params	SFT?		RL?		Scaffold	Base Model	Resolved%
			Process	Distillation	PRM	ORM			
SWE-GPT [140]	2024-11	72B	✓	✗	✗	✗	SWESynInfer [140]	Qwen2.5-Instruct	30.2
SWE-Gym [159]	2024-12	32B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-Coder-Instruct	32.0
ReSAT [143]	2024-12	7B	✓	✗	✗	✗	Agentless [236]	CodeQwen-1.5-Chat	7.2
SWE-Fixer [241]	2025-01	72B	✓	✗	✗	✗	SWE-Fixer [241]	Qwen2.5-Base	32.8
SWE-RL [233]	2025-02	70B	✗	✗	✗	✓	Agentless-Mini [233]	Llama3.3-Instruct	41.0
SoRFT [144]	2025-02	32B	✓	✓	✓	✗	Agentless [236]	Qwen2.5-Coder-Instruct	30.8
SWE-Reasoner [141]	2025-03	32B	✓	✓	✗	✓	SWESynInfer+ [141]	Qwen2.5-Coder-Instruct	46.0
SEAlign [273]	2025-03	14B	✗	✗	✗	✓	OpenHands [227]	Qwen2.5-Coder-Instruct	21.8
R2E-Gym [84]	2025-04	32B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-Coder-Instruct	34.4
SWE-agent-LM [254]	2025-04	32B	✗	✓	✗	✗	SWE-agent [252]	Qwen2.5-Coder-Instruct	40.2
Co-PatcheR [202]	2025-05	3*14B	✓	✓	✗	✗	PatchPilot [110]	Qwen2.5-Coder-Instruct	46.0
Satori-SWE [265]	2025-05	32B	✓	✓	✗	✓	Satori-SWE [265]	Qwen2.5-Coder-Instruct	41.6
Agent-RLVR [38]	2025-06	72B	✗	✗	✗	✓	Agentless [236]	Qwen2.5-Instruct	27.8
MCTS-Refine [229]	2025-06	72B	✓	✗	✗	✗	Agentless [236]	Qwen2.5-Instruct	35.0
SWE-Dev-LM [218]	2025-06	32B	✗	✓	✗	✓	OpenHands [227]	Qwen-2.5-Coder-Instruct	36.6
Skywork-SWE [267]	2025-06	32B	✗	✓	✗	✗	OpenHands [227]	Qwen-2.5-Coder-Instruct	38.0
DeepSWE [137]	2025-07	32B	✗	✗	✗	✓	DeepSWE [137]	Qwen3	59.0
RepoForge [33]	2025-08	8B	✗	✓	✗	✓	OpenHands [227]	Qwen3(Non-Thinking)	16.4
Golubev et al. [61]	2025-08	72B	✗	✗	✗	✓	Golubev et al. [61]	Qwen2.5-Instruct	39.0
Devstral-Small [170]	2025-08	24B	✗	✓	✗	✓	OpenHands [227]	Mistral Small 3	46.8
SWE-Swiss [76]	2025-08	32B	✓	✓	✗	✓	SWE-Swiss [76]	Qwen2.5-Instruct	60.2
SWE-Mirror-LM [220]	2025-09	32B	✗	✓	✗	✗	OpenHands [227]	Qwen-2.5-Coder-Instruct	52.2
Kimi-Dev [256]	2025-09	72B	✓	✓	✗	✓	Kimi-Dev [256]	Qwen2.5-Base	60.4
CWM [21]	2025-09	32B	✗	✓	✗	✓	CWM [21]	-	65.8
EntroPO [259]	2025-09	30B	✗	✓	✗	✓	R2E-Gym [84]	Qwen3-Coder	52.2
FrogBoss [192]	2025-10	32B	✗	✓	✗	✗	R2E-Gym [84]	Qwen3-32B	54.6
SWE-Play [288]	2025-12	32B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-Coder-32B-Instruct	31.2
SSR [234]	2025-12	32B	✗	✗	✗	✓	CWM [21]	CWM-sft [21]	51.4
SWE-Compressor [129]	2025-12	32B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-Coder-32B	57.6
SWE-RM [187]	2025-12	30B	✗	✗	✗	✓	OpenHands [227]	Qwen3-30B-A3B	54.8
SWE-Lego [203]	2026-01	32B	✗	✓	✗	✗	OpenHands [227]	Qwen3-32B	52.6
daVinci-Dev [266]	2026-01	72B	✗	✓	✗	✗	SWE-agent [252]	Qwen2.5-72B-Base	58.5
SERA [184]	2026-01	32B	✗	✓	✗	✗	SWE-agent [252]	Qwen3-32B	54.2
SWE-Spot [162]	2026-01	4B	✗	✓	✗	✗	mini-SWE-agent [252]	Qwen3-4B-Instruct	17.1
SWE-World [194]	2026-02	32B	✗	✓	✗	✓	R2E-Gym [84]	Qwen2.5-Coder-32B-Instruct	55.0
SWE-Master [189]	2026-02	32B	✗	✓	✗	✓	R2E-Gym [84]	Qwen2.5-Coder-32B-Instruct	61.4
SWE-Protégé [102]	2026-02	7B	✗	✓	✓	✓	SWE-agent [252]	Qwen2.5-Coder-7B-Instruct	42.4
SWE-Fuse [235]	2026-03	32B	✗	✓	✗	✓	mini-SWE-agent-plus [252]	Qwen3-32B	60.2
SWE-HERO [135]	2026-04	32B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-Coder-32B-Instruct	62.2
SWE-AGILE [118]	2026-04	8B	✗	✓	✗	✓	R2E-Gym [84]	Qwen3-8B	24.1
SWE-TRACE [71]	2026-04	30B	✗	✓	✓	✓	mini-SWE-agent [252]	Qwen3-30B-A3B	63.5
BoostAPR [117]	2026-05	32B	✗	✓	✗	✓	OpenHands [227]	Qwen2.5-Coder-32B-Instruct	40.7
HHD [223]	2026-05	72B	✗	✓	✗	✗	OpenHands [227]	Qwen2.5-72B-Base	51.2

Just Accepted

Received 24 December 2025; revised 27 July 2026; accepted 12 September 2026

Just Accepted